

11. Bölüm

Nesne Yönelimli Programlama

11.1 Nesne Yönelimli Programlama Paradigması

Yapısal programlamada talimatlar (**statement**) art arda koda yazılarak programlama yapılır ve programların neler yaptığı bu talimatlar izlenerek anlaşılabilir. Talimatların zincirin halkaları gibi birbirinin peşi sıra yazılarak yapılan programlamaya **emreden programlama** (**imperative programming**) paradigması adı verilir. Bu paradigmaya sahip diller, yazılımı yapılacak sürece ilişkin nelerin yapılacağını değil, işin nasıl yapılacağını belirtirler.

Emreden programlamanın bir örneği olan yapısal programlamada talimatlar yine art arda koda yazılarak programlama yapılır. 1980'li yıllara kadar yazılım ihtiyaçları yapısal programlama ile çözülmüştür. Kodlar büyüdükçe sorunlar artmış ve 2.8 başlığı altında anlatılan problemler ortaya çıkmıştır. Bu yıllarda Simula ve Smalltalk yaklaşımı yazılımcıların imdadına yetişmiştir. Bu dillerde **nesneler** (**object**) programın temel yapıtaşlarıdır. Nesneler; **durum** (**state**) ve **davranışlara** (**behavior**) sahiptir. Programlama, nesnelerin birbirlerine **ileti göndermesiyle** (**message-passing**) yapılır.

1979 senesinde bir Danimarkalı bilgisayar bilim adamı olan *Bjarne Stroustrup*, sonradan C++ olarak bilinecek olan sınıflarla C (**C with classes**) üzerinde çalışmaya başladı ve 1985 yılında ilk sürümünü yayınladı. C++ Programlama Dili;

- ◊ C Dilinden türetilmiştir.
- ◊ Düşük düzey programa yapılabilir
- ◊ İcra hızı yüksektir.
- ◊ C dilinden daha fazla kütüphaneye sahiptir.
- ◊ Nesne yönelimli programlamayı destekler.
- ◊ Diğer dillere göre daha etkin hafıza yönetimi sağlar
- ◊ **Standart Şablon Kütüphanesi** STL (**Standart Template Library**) ile jenerik (**generic**) programlamayı destekler.
- ◊ Birçok kavramı bir anda özümsemek gerektiğinden öğrenme eğrisi diktir.

Emreden paradigmanın aksine nesnelerin birbirlerine ileti göndermesi bakış açısıyla yapılan programlamaya **nesne yönelimli programlama** (**object oriented programming**) paradigması adı verilir.

11.2 Sınıf ve Nesne Nedir? Nasıl Tanımlanır?

Gerçek dünyamızda nesneleri sınıflamaya tabi tutarız. Örneğin bir birine benzeyen at, eşek ve zebra gibi hayvanları daha çocukluktan itibaren ayırt edebilir. İşte kafamızda oluşan bu mekanizmaya **sınıflayıcı** (**classifier**) adı verilir. Bu mekanizma ile sınıflanmış her bir türün gerçek dünyadaki her birine ise **örnek** (**instance**) adı verilir.

Nasıl ki dünyamızda evler bir mimari plan (**blueprint**) üzerinden inşa ediliyor, yazılımda kullanacağımız **nesneler** (**object**) de bir plan üzerinden inşa edilir. Nesnelere ilişkin verilerin tutulduğu **durumların** (**state**) ve nesnelerin göstereceği **davranışlarının** (**behavior**) tanımlandığı bu plana **sınıf** (**class**) adı verilir. Nesneler (**object**) yukarıda anlatılan gerçek dünya **örneklerine** (**instance**) karşılık gelir. Sınıflar ise **sınıflayıcıya** (**classifier**) karşılık gelir.

Nesne İmalatı

Bir mimari plandan birçok ev yapılabileceği gibi, **bir sınıftan da birçok nesne imal edilebilir.**

Gerçek dünyadaki nesneleri modelleyen **kullanıcı tanımlı sınıflar** (**user defined class**), aşağıdaki şekilde tanımlanır;

```
class sınıf-kimliği {  
    veri-tipi alan-değişkeni-kimliği; // field variable: alan değişkeni  
    veri-tipi davranış-kimliği(); // method: yöntem  
} [örnek-değişkeni-1][, örnek-değişkeni-2];
```

İmal edilecek nesnelerin durumlar (**state**), sınıftaki alanlarla (**field**) tanımlanır. Nesnelerin göstereceği davranışlar da sınıf içinde fonksiyon gibi tanımlanır. **Aynı davranış, farklı yöntemlerle yerine getirilebileceğinden**, davranışları tanımlayan bu fonksiyonlara **yöntem (method)** adı verilir. İşin doğası gereği davranışlar durumlardan etkilenirler.

Sınıf

Özet olarak sınıflar, kodumuzda **imal edilecek nesnelere ilişkin ortak davranış ve durumları tasnif eden bir şablon bileşenidir**.

Aşağıda ortak davranış ve özelliklerin tanımlandığı bir **Ogrenci** sınıfı örneği verilmiştir;

```
1 class Ogrenci {
2 public:
3     unsigned yas; // alan(field)
4     char cinsiyet; // alan(field)
5     void yasiniSoyle() { // yöntem (method)
6         std::cout << "Yaşım:" << yas << std::endl;
7     }
8     void cinsiyetSoyle() { // yöntem (method)
9         if (cinsiyet=='E' || cinsiyet=='e')
10            std::cout << "Cinsiyetim: ERKEK" << std::endl;
11        else if (cinsiyet=='K' || cinsiyet=='k')
12            std::cout << "Cinsiyetim: KADIN" << std::endl;
13        else
14            std::cout << "Cinsiyetim: SÖYLEMEK İSTEMİYORUM!" << std::endl;
15    }
16 } ilhan, methet;
```

Bu tanımlamayı şu şekilde açıklayabiliriz;

- ◇ 1. Satırda **class** anahtar kelimesi ile **Ogrenci** kimlikli bir sınıf tanımlanacağı belirtilmiştir. Sınıfın durum ve davranışları tarif etmek için blok açılmıştır. Kod bloğu olmadan sınıf tanımlanmaz!
- ◇ 2. Satırda **public:** etiketinden sonra tanımlanacak davranış ve özelliklerinin yer alacağı kısım başlayacağı belirtilmiştir. Bu kısımda sınıftan imal edilecek nesnelerin umuma açık (**public**) durum (**state**) ve davranışlarıdır (**behavior**). Bir sonraki başlıkta anlatılmıştır.
- ◇ 3. Satırda tanımlanan **yas** kimlikli alan (**field**), imal edilecek nesnelerin yaşına ait durum (**state**) verilerini tutar.
- ◇ 4. Satırda tanımlanan **cinsiyet** kimlikli alan (**field**), imal edilecek nesnelerin cinsiyetine ait durum (**state**) verilerini tutar.
- ◇ 5. Satırda başlayan ve 7. satırda biten **yasiniSoyle()** yöntemi (**method**), imal edilecek nesnelerin yaşını söyleme davranışını (**behavior**) tanımlıyor. Davranışlar 2.7 başlığında anlatılan yapısal programlamadaki fonksiyonlar gibi tanımlanır.
- ◇ 8. Satırda başlayan ve 15. satırda biten **cinsiyetSoyle()** yöntemi (**method**), imal edilecek nesnelerin cinsiyetini söyleme davranışını (**behavior**) tanımlıyor.
- ◇ 16. Satırda tanımlanan sınıfa ait açılan blok kapatılıyor ve **Ogrenci** sınıfından **ilhan** ve **mehmet** nesneleri imal edildi. Son olarak tanımlama tamamlandığından noktalı virgül karakteri ile talimat tamamlanıyor. Nesne imal edilmese bile kullanılacaktı.

Tanımlanmış bir sınıftan değişken tanımlar gibi **sınıf-kimliği örnek-değişkeni**; şeklinde yeni nesneler imal edilebilir. İmal edilen nesnenin durum ve davranışlarına **nokta işleci (dot operator)** ile erişilir.

```
#include <iostream>
class Ogrenci {
public:
    unsigned yas; // alan(field)
    char cinsiyet; // alan(field)
    void yasiniSoyle() { // yöntem (method)
        std::cout << "Yaşım:" << yas << std::endl;
    }
    void cinsiyetSoyle() { // yöntem (method)
        if (cinsiyet=='E' || cinsiyet=='e')
            std::cout << "Cinsiyetim: ERKEK" << std::endl;
        else if (cinsiyet=='K' || cinsiyet=='k')
            std::cout << "Cinsiyetim: KADIN" << std::endl;
    }
}
```

```

        else
            std::cout << "Cinsiyetim: SÖYLEMEK İSTEMİYORUM!" << std::endl;
    }
} ilhan, methet;
int main() {
    std::cout << "ilhan nesnesi durum ve davranışları:" << std::endl;
    ilhan.yas=50; // Sınıf tanımı ile birlikte tanımlanmış "ilhan" nesnesinin "yas" özelliği
    ↪ değiştirildi.
    ilhan.yasiniSoyle(); /* "ilhan" nesnesine yasiniSoyle() iletisi gönderildi (message passing).
    ↪ */
    ilhan.cinsiyet='E';
    ilhan.cinsiyetSoyle(); /* "ilhan" nesnesine cinsiyetSoyle() iletisi gönderildi (message
    ↪ passing).*/

    Ogrenci damla; // Ogrenci sınıfından "damla" nesnesi imal edildi
    std::cout << "damla nesnesi durum ve davranışları:" << std::endl;
    damla.yas=30; // "damla" nesnesinin "yas" özelliği değiştirildi.
    damla.cinsiyet='K';
    damla.yasiniSoyle(); /* "damla" nesnesine yasiniSoyle() iletisi gönderildi (message passing).
    ↪ */
    damla.cinsiyetSoyle(); /* "damla" nesnesine cinsiyetSoyle() iletisi gönderildi (message
    ↪ passing). */
}
/* Program çalıştırıldığında:
ilhan nesnesi durum ve davranışları:
Yaşım:50
Cinsiyetim: ERKEK
damla nesnesi durum ve davranışları:
Yaşım:30
Cinsiyetim: KADIN
*/

```

Burada imal edilen nesnelere ilişkin durumlarının, davranışları etkilediği gözden kaçırılmamalıdır. İmal edilen `ilhan` nesnesinin `cinsiyetiniSoyle()` davranışı ve `damla` nesnesinin `cinsiyetiniSoyle()` davranışı birbirinden farklıdır. Çünkü `cinsiyet` durumu, her iki nesnede farklıdır.

Nesne Durumları

Nesne durumlarına ilişkin verileri tutacak değişkenler, sınıf tanımındaki alanlarla (**field**) tanımlanır. **Bu alanlar, her nesne için tahsis edilen bellek bölgesinde nesneye özel değişkenler haline gelir.**

Yukarıda verilen `Ogrenci` sınıfının yöntemlerini mutlaka sınıf içinde tanımlamak gerekmez. Kapsam çözümleme işleci (**scope resolution operator**) kullanılarak sınıf dışında da tanımlanabilir:

```

#include <iostream>
class Ogrenci {
public:
    unsigned yas; // alan(field)
    char cinsiyet; // alan(field)
    void yasiniSoyle(); //Yöntem sınıf dışında tanımlanmalı!
    void cinsiyetSoyle(); //Yöntem sınıf dışında tanımlanmalı!
};
void Ogrenci::yasiniSoyle() {
    std::cout << "Yaşım:" << yas << std::endl;
}
void Ogrenci::cinsiyetSoyle() { // yöntem (method)
    if (cinsiyet=='E' || cinsiyet=='e')
        std::cout << "Cinsiyetim: ERKEK" << std::endl;
    else if (cinsiyet=='K' || cinsiyet=='k')
        std::cout << "Cinsiyetim: KADIN" << std::endl;
    else
        std::cout << "Cinsiyetim: SÖYLEMEK İSTEMİYORUM!" << std::endl;
}

```

```
int main() {
    Ogrenci ilhan;
    ilhan.yas=45;
    ilhan.cinsiyet='e';
    ilhan.yasiniSoyle();
    ilhan.cinsiyetSoyle();
}
/*Program Çıktısı:
Yaşım:45
Cinsiyetim: ERKEK
*/
```

Sınıf Mimarisi ve Bellek Hizalaması

Yapılarda olduğu gibi bir sınıfın (**class**) bellekte ne kadar yer kapladığını (**sizeof**) anlatırken bellek hizalaması (**padding**) yapılır. Örneğin; bir sınıf içinde sırasıyla **char** (1 bayt), **int** (4 bayt) ve **char** (1 bayt) alan değişkenleri tanımlandığında toplam boyutun 6 bayt değil de işlemci mimarisine göre 12 bayt çıkar.

11.3 Erişim Değiştiricileri

Sınıf üyelerinin (alan değişkeni veya yöntem) başka nesne ve sınıfların nasıl erişeceğini **erişim değiştiricileri** (**access modifier**) belirler. **Görünürlük** (**visibility**) sıfatları olarak da adlandırılan bu değiştiriciler, sınıf üyelerine sınıf içinden ve dışından erişimi belirleyen sıfatlardır.

Erişim Değiştiricileri

- ◇ **public**: Sınıf üyesine, sınıf dışından her zaman erişilebilir. **Umuma açık** (**public**) davranış ve özellikleri için bu belirleyici kullanılır.
- ◇ **private**: Sınıf üyesine, sınıf dışından erişime hiçbir zaman izin verilmez. **Mahrem/şahsi/kişisel** davranış ve özellikleri için bu belirleyici kullanılır. Varsayılan (**default**) erişim değiştiricidir. Bir sınıfta hiçbir erişim değiştirici yoksa tanımlanan durum ve davranışlar **mahrem** (**private**) olarak kabul edilir.
- ◇ **protected**: Dış dünyaya kapalı (**private**), ama mirasçılara açık sınıf üyeleri için **korumalı** (**protected**) belirleyicisi kullanılır. Miras alma konusu ileride anlatılacaktır.

Aşağıda erişim belirleyicilerin uygulandığı basit bir **Kisi** sınıfı örnek verilmiştir. Bu sınıftan imal edilen nesnelerin **yas** durumu diğer tüm nesneler tarafından erişilip değiştirilebilir, ancak **sir** durumuna ise hiçbir nesne tarafından erişilemez. **maas** özelliğine ise **Kisi** sınıfı ve bu sınıftan türemiş sınıflardan imal edilen nesneler tarafından bu özelliğe erişilebilir. İleride göreceğiz.

```
class Kisi {
public: /* imal edilecek nesnelerin umuma açık (public) davranış ve özellikleri:*/
    unsigned yas; /* "yas" kimlikli alan(field), ogrencinin yaşına ait durumlara(state) ilişkin
        ↳ verileri tutar. İmal edilecek nesnelerde umuma açık bir özelliktir. */
private:/* imal edilecek nesnelerin mahrem (private) davranış ve özellikleri:*/
    int sir; /* "sir" kimlikli alan(field), ogrencinin sırrına ait durumlara(state) ilişkin
        ↳ verileri tutar. İmal edilecek nesnelerde mahrem bir özelliktir. diğer nesneler/programlar
        ↳ tarafından bu özelliğe erişilemez. */
protected: /* bu sınıftan ve türetilmiş sınıflardan imal edilecek nesnelerin korumalı (protected)
        ↳ davranış ve özellikler: */
    float maas; /* "maas" kimlikli alan(field), ogrencinin maaşına ait durumlara(state) ilişkin
        ↳ verileri tutar. imal edilecek nesnelerde mahrem bir özelliktir. Ogrenci sınıfından
        ↳ türemiş sınıflar (ileride göreceğiz) dışında diğer nesneler/programlar tarafından bu
        ↳ özelliğe erişilemez.*/
}; //Hiç nesne tanımlanmamış! Noktalı virgül ile bitmiş!

int main() {
    Kisi ilhan; //Kisi sınıfından "ilhan" nesnesi imal edildi
    ilhan.yas=50; // "ilhan" nesnesinin "yas" özelliği değiştirildi.
    //ilhan.sir=-1; //HATA: Ana program, ilhan nesnesinin sir özelliğine ulaşamaz.
    //ilhan.maas=10000.0; //HATA: Ana program, ilhan nesnesinin maas özelliğine ulaşamaz.
```

}

11.4 Nesne Yapıcısı

Bir sınıftan birçok nesne de imal edilebileceği anlatılmıştı. İmal edilen her nesne (**object**) sınıfın bir örneğidir (**instance**) ve bu nesnelerin varsayılan (**default**) olarak belirlenen durumlarla (**state**) imal edilmesi gerekiyorsa **nesne yapıcısı** (**constructor**) kullanılır.

Nesne Yapıcıları

- Yapıcıların görevi bir sınıftan nesneleri, **varsayılan durumlara yani verilere sahip olarak imal etmektir**.
- Nesnelere ait veriler, alanlarda (**field**) tutulurlar ve nesnenin durumları (**state**), nesnenin davranışını (**behavior**) etkiler! Bu nedenle daha nesneler imal edilirken durumlarının belirlenen değerde olması istenir.
- Yapıcıların kimliği, sınıf kimliğiyle aynıdır.**
- Yapıcılar, yöntemlerin aksine **geri değer döndürmezler**. Bu nedenle bir yapıcı içerisinde **return** talimatı da kullanılmaz.

Aşağıda verilen örnekte **Ogrenci** sınıfından imal edilecek her nesnenin yaşı 6 ve cinsiyeti 'E' olarak üretilecektir. Hiçbir parametresi olmayan bu yapıcıya **varsayılan yapıcı** (**default constructor**) adı verilir.

```
#include <iostream>
class Ogrenci {
public: /* Umuma açık (public) davranış ve özellikler: */
    Ogrenci() { // Ogrenci sınıfı yapıcısı: varsayılan yapıcı
        // Yapıcı içerisinde, imal edilecek nesnelere ilk değer verilir.
        yas=6;
        cinsiyet='E';
        // Yapıcı içerisinde return talimatı bulunmaz!
    }
    unsigned yas; /* "yas" kimlikli alan(field) */
    char cinsiyet; /* "cinsiyet" kimlikli alan(field),*/
    void yasiniSoyle() { /* yasiniSoyle() yöntemi(method) */
        std::cout << "Yaşım:" << yas << std::endl;
    }
    void cinsiyetSoyle() { /* cinsiyetSoyle() yöntemi(method)*/
        if (cinsiyet=='E' || cinsiyet=='e')
            std::cout << "Cinsiyetim: ERKEK" << std::endl;
        else if (cinsiyet=='K' || cinsiyet=='k')
            std::cout << "Cinsiyetim: KADIN" << std::endl;
        else
            std::cout << "Cinsiyetim: SÖYLEMEK İSTEMİYORUM!" << std::endl;
    }
};
int main() {
    Ogrenci ilhan; /* Ogrenci sınıfından "ilhan" nesnesi; Ogrenci() varsayılan yapıcısı
        → kullanılarak yaşı 6 ve cinsiyeti E olarak imal edildi. */
    ilhan.yasiniSoyle(); /* "ilhan" nesnesine yasiniSoyle() iletisi gönderildi (message passing).
        → */
    ilhan.cinsiyetSoyle(); /* "ilhan" nesnesine cinsiyetSoyle() iletisi gönderildi (message
        → passing).*/
    std::cout << "----" << std::endl;
    ilhan.yas=50; // "ilhan" nesnesinin "yas" özelliği değiştirildi.
    ilhan.yasiniSoyle(); /* "ilhan" nesnesine yasiniSoyle() iletisi gönderildi (message passing).
        → */
}
/*Program Çıktısı:
Yaşım:6
Cinsiyetim: ERKEK
----
Yaşım:50
*/
```

Nesne imal edilirken belirlen durumlarda nesne imal etmek istenebilir. Bu durumda **parametrelili yapıcılar** (**parameterized constructor**) kullanılır;

```
#include <iostream>
class Ogrenci {
public: /* Umuma açık (public) davranış ve özellikler: */
    Ogrenci() { // Ogrenci sınıfı varsayılan yapıcısı (default constructor)
        yas=6;
        cinsiyet='E';
    }
    Ogrenci(int pYas, char pCinsiyet): yas(pYas), cinsiyet(pCinsiyet) { // Ogrenci sınıfı
        ↳ parametrelili yapıcısı (parameterized constructor)
    }
    /*
    // Parametrelili yapıcı, alan değişkenlerine blok içinde ilk değer verilecek şekilde
    ↳ tanımlanabilirdi:
    Ogrenci(int pYas, char pCinsiyet) {
        yas=pYas;
        cinsiyet=pCinsiyet;
    }
    */
    unsigned yas; /* "yas" kimlikli alan(field) */
    char cinsiyet; /* "cinsiyet" kimlikli alan(field) */
    void yasiniSoyle() { /* yasiniSoyle() yöntemi(method) */
        std::cout << "Yaşım:" << yas << std::endl;
    }
    void cinsiyetSoyle() { /* cinsiyetSoyle() yöntemi(method) */
        if (cinsiyet=='E' || cinsiyet=='e')
            std::cout << "Cinsiyetim: ERKEK" << std::endl;
        else if (cinsiyet=='K' || cinsiyet=='k')
            std::cout << "Cinsiyetim: KADIN" << std::endl;
        else
            std::cout << "Cinsiyetim: SÖYLEMEK İSTEMİYORUM!" << std::endl;
    }
};

int main() {
    Ogrenci ilhan; /* Ogrenci sınıfından "ilhan" nesnesi; Ogrenci() varsayılan yapıcısı
    ↳ kullanılarak yaşı 6 ve cinsiyeti E olarak imal edildi. */
    ilhan.yasiniSoyle();
    ilhan.cinsiyetSoyle();
    std::cout << "----" << std::endl;
    Ogrenci fatma(45,'K'); /* Ogrenci sınıfından "fatma" nesnesi; Ogrenci(int pYas, char
    ↳ pCinsiyet) parametrelili yapıcısı kullanılarak yaşı 6 ve cinsiyeti E olarak imal edildi. */
    fatma.yasiniSoyle();
    fatma.cinsiyetSoyle();
}

/*Program Çıktısı:
Yaşım:6
Cinsiyetim: ERKEK
----
Yaşım:45
Cinsiyetim: KADIN
*/
```

Bazı durumlarda bir yapıcı bir başka yapıcıyı çağırabilir. Bu durumda **devredilen yapıcı** (**delegating constructor**) kullanılır.

```
#include <iostream>
class Ogrenci {
public: /* Umuma açık (public) davranış ve özellikler: */
    Ogrenci() { // Ogrenci sınıfı varsayılan yapıcısı (default constructor)
        yas=6;
        cinsiyet='E';
    }
}
```

```

Ogrenci(int pYas) { // Ogrenci sınıfı parametrelili yapıcısı (parameterized constructor)
    yas=pYas;
}
Ogrenci(int pYas, char pCinsiyet): Ogrenci(pYas) { // Devredilen yapıcı (delegated
    → constructor)
    cinsiyet=pCinsiyet;
}
unsigned yas; /* "yas" kimlikli alan(field) */
char cinsiyet; /* "cinsiyet" kimlikli alan(field) */
void yasiniSoyle() { /* yasiniSoyle() yöntemi(method) */
    std::cout << "Yaşım:" << yas << std::endl;
}
void cinsiyetSoyle() { /* cinsiyetSoyle() yöntemi(method) */
    if (cinsiyet=='E' || cinsiyet=='e')
        std::cout << "Cinsiyetim: ERKEK" << std::endl;
    else if (cinsiyet=='K' || cinsiyet=='k')
        std::cout << "Cinsiyetim: KADIN" << std::endl;
    else
        std::cout << "Cinsiyetim: SÖYLEMEK İSTEMİYORUM!" << std::endl;
}
};
int main() {
    Ogrenci ilhan; /* Ogrenci sınıfından "ilhan" nesnesi; Ogrenci() varsayılan yapıcısı
    → kullanılarak yası 6 ve cinsiyeti E olarak imal edildi. */
    ilhan.yasiniSoyle();
    ilhan.cinsiyetSoyle();
    std::cout << "----" << std::endl;
    Ogrenci ayse(30); /* Ogrenci sınıfından "ayse" nesnesi; Ogrenci(int pYas) yetkilendirilmiş
    → yapıcı kullanılarak yası 6 olarak imal edildi. */
    ayse.yasiniSoyle();
    ayse.cinsiyetSoyle();
    std::cout << "----" << std::endl;
    Ogrenci fatma(45,'K'); /* Ogrenci sınıfından "fatma" nesnesi; Ogrenci(int pYas, char
    → pCinsiyet) parametrelili yapıcısı kullanılarak yası 6 ve cinsiyeti E olarak imal edildi. */
    fatma.yasiniSoyle();
    fatma.cinsiyetSoyle();
}
/*Program Çıktısı:
Yaşım:6
Cinsiyetim: ERKEK
----
Yaşım:30
Cinsiyetim: SÖYLEMEK İSTEMİYORUM!
----
Yaşım:45
Cinsiyetim: KADIN
*/

```

11.5 Dinamik Nesne İmalatı

Değişkenleri çalışma anında üretip dinamik olarak kullanma 9.3 başlığında anlatılmıştı . Nesneleri de çalışma anında da yapıcılar (**constructor**) kullanarak imal ederiz ve buna **dinamik başlatma** (**dynamic initialization**) adı verilir. Dinamik başlatma aşağıdaki şekilde yapılır;

```
SınıfKimliği* imal-edilen-nesne-göstericisi = new SınıfKimliği(yapıcı-argümanları);
```


Dinamik Nesne İmalatı

- ◊ Dinamik olarak **new işleci** (**new operator**) ile imal edilecek nesneye öbek bellekte (**heap segment**) yer tahsis edilir (**memory allocation**). Ardından nesnenin yapıcısı ile veri tutan alanlarına (**field**) ilk değer değerler için mantık çalıştırılır ve nesne göstericisi geri döner.
- ◊ Nesne göstericileri üzerinden sınıf üyelerine **dolaylı işleç** (**indirection operator**) (**->**) ile erişilir. Bu işleç, dinamik olmayan nesneler için kullanılan nokta işleci (**dot operator**) ile aynı önceliklidir.
- ◊ Dinamik olarak imal edilmiş nesne kullanıldıktan sonra **delete işleci** (**delete operator**) ile bellekten kaldırılabilir (**deallocate memory**).

Çeşitli yapıcılara sahip, dinamik olarak nesne imal edilen bir kod örneği aşağıda verilmiştir;

```
#include <iostream>
class Karmasik {
public:
    float gercek, sanal;
    // 1. Varsayılan Yapıcı: Üye ilklendirme listesi kullanımı daha performanslıdır.
    Karmasik() : gercek(0.0f), sanal(0.0f) {
        std::cout << "Varsayılan (default) yapici cagrildi" << std::endl;
    }
    // 2. Parametrelili Yapıcı
    Karmasik(float pGercek, float pSanal) : gercek(pGercek), sanal(pSanal) {
        std::cout << "İki Parametrelili yapici cagrildi" << std::endl;
    }
    // 3. Devredilen Yapıcı (Delegating Constructor)
    Karmasik(float pGercek) : Karmasik(pGercek, 0.0f) {
        std::cout << "Tek parametrelili yapici cagrildi. (İki parametreliliye devredildi)" <<
            << std::endl;
    }
    // 'const' ekleyerek metodun nesne üzerinde değişiklik yapmayacağını garanti ediyoruz.
    void yaz() const {
        // Kodundaki "+"i kısmındaki fazla "+" işaretini kaldırdım.
        std::cout << gercek << " + " << sanal << "i";
    }
};

int main() {
    std::cout << "--- Nesne 1 ---" << std::endl;
    Karmasik* k1 = new Karmasik(); /* Dinamik olarak bir Karmasik nesnesi imal edildi. Nesneye
    ↪ yer tahsis edildikten sonra varsayılan yapıcı Karmasik() ile başlatma mantığı çağrıldı */
    k1->yaz();
    std::cout << "\n--- Nesne 2 ---" << std::endl;
    Karmasik* k2 = new Karmasik(2.0f, 3.0f); /* Yine Dinamik olarak bir Karmasik nesnesi imal
    ↪ edildi. Nesneye yer tahsis edildikten sonra Karmasik(float pGercek, float pSanal)
    ↪ yapıcısı ile başlatma mantığı çağrıldı */
    k2->yaz();
    std::cout << "\n--- Nesne 3 ---" << std::endl;
    Karmasik* k3 = new Karmasik(4.0f); /* Bir başka Dinamik olarak bir Karmasik nesnesi imal
    ↪ edildi. Nesneye yer tahsis edildikten sonra Karmasik(float pGercek) yapıcısı ile başlatma
    ↪ mantığı çağrıldı */
    k3->yaz();
    // Bellek temizliği
    delete k1;
    delete k2;
    delete k3;
    // Modern C++'ta new/delete yerine genellikle stack (yığın) nesneleri tercih edilir.
    // Bunun yerine daha güvenli olan ileride anlatacağımız Akıllı göstericiler (smart pointers)
    ↪ kullanılmalıdır.
}

/* Program Çıktısı:
--- Nesne 1 ---
Varsayılan (default) yapici cagrildi
0 + 0i
```



```

--- Nesne 2 ---
İki Parametreli yapıcı cagrildi
2 + 3i
--- Nesne 3 ---
İki Parametreli yapıcı cagrildi
Tek parametreli yapıcı cagrildi. (İki parametreliliye devredildi)
4 + 0i
*/

```

11.6 this Göstericisi

C++'da her nesne, **this** adı verilen önemli bir gösterici aracılığıyla kendi adresine erişebilir. **this** Göstericisi (**this pointer**), tüm üye yöntemler için örtük bir değişkendir. Aşağıda karmaşık sayılar örneği verilmiştir;

```

#include <iostream>
class Karmasik {
    float gercek, sanal;
public:
    Karmasik(float pGercek=0.0, float pSanal=0.0) {
        this->gercek=pGercek;
        this->sanal=pSanal;
    }
    void yaz() { //yaz yöntemi
        std::cout<< this->gercek << "+" << this-> sanal << "+"i" << std::endl;
    }
};
int main(void) {
    Karmasik c1(1.0,2.0);
    c1.yaz();
    Karmasik c2(3.0,5.0);
    c2.yaz();
}

```

11.7 Soyut, Somut ve Bilgi Gizleme

Soyut (**abstract**) kavramı kelime itibariyle detay içermeyen, özet anlamına gelir. Soyutlama (**abstraction**), bir nesnenin sadece "ne yaptığını" ortaya koyup, "nasıl yaptığını" gizleme sürecidir. Kullanıcıya sadece gerekli olan temel özellikleri sunar, karmaşık detayları arka planda tutar. Örnek olarak bir **Araba** sınıfı düşünün. Kullanıcı bu sınıftan imal edilmiş bir nesneden **gazaBas()** davranışını göstermesi için ileti gönderebilir. Motorun içindeki yanma odalarında neler olduğu (detay), kullanıcının bilmesi gereken bir şey değildir (soyutlama).

Somut (**concrete**) ise en ince detayına kadar bilinen, elle tutulur anlamına gelir. **Somutlama** (**concretization**), soyut olarak tanımlanmış bir yapının gerçek hayattaki karşılığının oluşturulması veya ileride göreceğimiz bir arayüzün (**interface**) içinin doldurulmasıdır. Yani kodlayarak uygulamanın hayata geçirilmesidir (**implementation**).

Bilgi gizleme (**information hiding**), bir nesnenin iç durumuna (verilerine) dışarıdan doğrudan erişilmesini engellemek ve bu verileri koruma altına almaktır. Genellikle ileride göreceğimiz kapsülleme (**encapsulation**) ile yapılır.

Nesne Yapıcıları

Bir sınıfta **soyutlama** (**abstraction**) iki türlü yapılır; Birincisi **veri soyutlaması** (**data abstraction**), nesnenin durumları üzerinde yapılan soyutlamadır. İkincisi ise **kontrol soyutlaması** (**control abstraction**), nesnenin davranışları üzerinde yapılan soyutlamadır.

11.8 Kapsülleme

Yukarıda verilen **Ogrenci** örneğinden devam edecek olursak imal edilecek nesnenin **yas** durumuna bir başka nesne veya program, **-1** gibi bir değer atayabilir. Bu durumda **yasiniSoyle()** davranışında istenmeyen bir çıktı verilmesine sebep olur. Bunu gidermek için veri soyutlaması (**data abstraction**) yapılması gerekir. Bunun için **yas** alanına bir değer koymayı ve **yas** alanından bir değer almayı kontrol altına almalıyız. Benzer durum **cinsiyet** alanı için de geçerlidir.

Ogrenci sınıfında **veri soyutlamasını** yapmak için **yas** ve **cinsiyet** alanlarını **mahrem** (**private**) olarak

belirleyip bu alanlara değer atama ve değer okuma işlevlerini yerine getirecek `get` ve `set` yöntemlerini umuma açık (**public**) olarak tanımlamalıyız;

```
#include <iostream>
class Ogrenci {
private: // mahrem (private) davranış ve özellikler:
    unsigned yas; /* "yas" kimlikli alan(field) */
    char cinsiyet; /* "cinsiyet" kimlikli alan(field) */

public: // umuma açık (public) davranış ve özellikler:
    Ogrenci() { // Ogrenci yapıcısı
        yas=6;
        cinsiyet='E';
    }
    //VERİ SOYUTLAMASI İÇİN YÖNTEMLER:
    void setYas(unsigned yeniYas) {
        if (yeniYas <5)
            std::cout << "Öğrenci 5 Yaşından Küçük Olamaz." << std::endl;
        else
            yas=yeniYas;
    }
    unsigned getYas() {
        return yas;
    }
    void setCinsiyet(char yeniCinsiyet) {
        if (yeniCinsiyet == 'E' ||
            yeniCinsiyet == 'e' ||
            yeniCinsiyet == 'K' ||
            yeniCinsiyet == 'k' ||
            yeniCinsiyet == 'B' ||
            yeniCinsiyet == 'b')
            cinsiyet=yeniCinsiyet;
        else
            std::cout << "Cinsiyet E,e,K,k,B,b Dışında Bir Değer Olamaz." << std::endl;
    }
    char getCinsiyet() {
        return cinsiyet;
    }
    //DAVRANIŞLAR:
    void yasiniSoyle() { /* yasiniSoyle() yöntemi(method) */
        std::cout << "Yaşım:" << yas << std::endl;
    }
    void cinsiyetSoyle() { /* cinsiyetSoyle() yöntemi(method) */
        if (cinsiyet=='E' || cinsiyet=='e')
            std::cout << "Cinsiyetim: ERKEK" << std::endl;
        else if (cinsiyet=='K' || cinsiyet=='k')
            std::cout << "Cinsiyetim: KADIN" << std::endl;
        else
            std::cout << "Cinsiyetim: SÖYLEMEK İSTEMİYORUM!" << std::endl;
    }
};

int main() {
    Ogrenci ilhan; /* Ogrenci sınıfından imal edilen "ilhan" nesnesi*/
    ilhan.yasiniSoyle(); /* "ilhan" nesnesine yasiniSoyle() iletisi gönderildi (message passing).
    ↪ */
    ilhan.cinsiyetSoyle(); /* "ilhan" nesnesine cinsiyetSoyle() iletisi gönderildi (message
    ↪ passing).*/

    //ilhan.yas=12; // Bu talimat çalışmaz. Çünkü yaz alanı private

    ilhan.setYas(0); // "ilhan" nesnesinin "yas" özelliği değiştirildi.
    ilhan.yasiniSoyle(); /* "ilhan" nesnesine yasiniSoyle() iletisi gönderildi (message passing).
    ↪ */
}
```

```

        ilhan.setCinsiyet('H');
        ilhan.cinsiyetSoyle();
    }
    /*Program Çıktısı:
    Yaşım:6
    Cinsiyetim: ERKEK
    Öğrenci 5 Yaşından Küçük Olamaz.
    Yaşım:6
    Cinsiyet E,e,K,k,B,b Dışında Bir Değer Olamaz.
    Cinsiyetim: ERKEK
    */

```

Özellik

İşte bir durumun (**state**) veri soyutlaması yapılmış haline **özellik (property)** adı veririz. Çünkü artık duruma dışarıdan doğrudan müdahale edilemez. Nesnenin **get** ve **set** yöntemleri ile durumları üzerinde kontrol hakları vardır.

Benzer şekilde davranışlar (**behavior**) da nesne dışına kontrollü olarak açılabilir. Bu durumda da **kontrol soyutlaması (control abstraction)** yapılır. Aşağıda faktöriyel hesaplayan bir hesap makinesinin ekranı yenileme mahrem davranışı buna örnek verilebilir;

```

#include <iostream>
class HesapMakinesi {
private: // mahrem (private) davranış ve özellikler:
    int hesaplanan;
    void ekraniYenile() {
        std::cout << "Hesaplanan:" << hesaplanan << std::endl;
    }
public: // umuma açık (public) davranış ve özellikler:
    HesapMakinesi() { //Yapıcı
        hesaplanan=0;
        ekraniYenile();
    }
    void setHesapMakinesi(int pSayi) {
        hesaplanan=pSayi;
        ekraniYenile();
    }
    int getHesapMakinesi() {
        return hesaplanan;
    }
    void makineyiSifirla() {
        hesaplanan=0;
        ekraniYenile();
    }
    void FaktoriyelHesapla() {
        unsigned temp=1;
        for (int sayac = hesaplanan; sayac > 0; sayac--)
            temp = temp*sayac;
        hesaplanan=temp;
        ekraniYenile();
    }
};
int main ()
{
    HesapMakinesi hesaplayici; // hesaplayici nesnesi imal edildi
    hesaplayici.setHesapMakinesi(4);
    hesaplayici.FaktoriyelHesapla();
    hesaplayici.setHesapMakinesi(6);
    hesaplayici.FaktoriyelHesapla();
    hesaplayici.makineyiSifirla();
}
/*Program Çıktısı:

```

```
Hesaplanan:0
Hesaplanan:4
Hesaplanan:24
Hesaplanan:6
Hesaplanan:720
Hesaplanan:0
*/
```

Kapsülleme

Bir sınıf üzerinde veri soyutlaması (**data abstraction**) yapılarak veriler üzerine bir kılıf (**capsule**) çekilir, bir de kontrol soyutlaması (**control abstraction**) ile davranışların üzerine bir kılıf daha (**capsule**) çekilir. **Bir sınıf üzerinde her iki soyutlamanın yapılmasına kapsülleme (**encapsulation**) adı verilir.**

Soyutlamaların gerekli olup olmadığına programcı karar verir. Kapsülleme işlemi örneklerde görüldüğü üzere erişim değiştiricilerle (**access modifier**) yapılır. Böylece imal edilen nesnelerin verileri ve davranışları güvenli bir şekilde dış dünyaya açılmış, istenmeyen detaylar dış dünyadan gizlenir (**information hiding**). Böylece, sınıf ve nesne gibi bileşenlerin içeriğini bilmeden soyut (**abstract**) olarak kullanmak mümkün olmaktadır. Programcı, tasarladığı sınıfın soyut olarak kullanılacağını düşünerek sınıfları tasarlamalıdır.

Kapsülleme (encapsulation**)** tekniğini uygulamak, **kullanımı kolay** sınıf (**class**), bileşen (**component**), kütüphane (**library**) ve çerçeve yapılar (**framework**) elde etmemizi sağlar;

- ◊ n adet durum/veri (**state/data**) ve n adet davranış (**behavior**) kapsülleme işlemine tabi tutulursa sınıf,
- ◊ n adet sınıf kapsülleme işlemine tabi tutulursa bileşen,
- ◊ n adet bileşen kapsülleme işlemine tabi tutulursa kütüphane,
- ◊ n adet kütüphane kapsülleme işlemine tabi tutulursa çerçeve yapılar oluşur.

Kapsüllemenin Üstünlüğü

Kapsülleme, projemizde proje süresini ve maliyetini kısaltma üstünlüğünü verir.

11.9 Kalıtım

Öğrenci, öğretmen ve müdürün olduğu bir sistemi örnek olarak ele alalım. Bunlardan her birinden nesne imal etmek için her birine ayrı ayrı sınıf tanımlamak gerekir. Öğrenci için bir sınıf;

```
class Ogrenci {
private:
    unsigned yas;
    char cinsiyet;
    unsigned ogrenciNo;
public:
    Ogrenci() {
        yas=18u;
        cinsiyet='E';
        ogrenciNo=0u;
    }
    void setYas(unsigned yeniYas) {
        if (yeniYas <18)
            std::cout << "Yaş 18 den Küçük Olamaz." << std::endl;
        else
            yas=yeniYas;
    }
    unsigned getYas() {
        return yas;
    }
    void setCinsiyet(char yeniCinsiyet) {
        if (yeniCinsiyet == 'E' || yeniCinsiyet == 'E' || yeniCinsiyet == 'K' || yeniCinsiyet ==
            ↪ 'k' || yeniCinsiyet == 'B' || yeniCinsiyet == 'b')
            cinsiyet=yeniCinsiyet;
        else
            std::cout << "Cinsiyet E,e,K,k,B,b Dışında Bir Değer Olamaz." << std::endl;
    }
}
```

```

    }
    char getCinsiyet() {
        return cinsiyet;
    }
    void yasiniSoyle() {
        std::cout << "Yaşım:" << yas << std::endl;
    }
    void cinsiyetSoyle() {
        if (cinsiyet=='E' || cinsiyet=='e')
            std::cout << "Cinsiyetim: ERKEK" << std::endl;
        else if (cinsiyet=='K' || cinsiyet=='k')
            std::cout << "Cinsiyetim: KADIN" << std::endl;
        else
            std::cout << "Cinsiyetim: SÖYLEMEK İSTEMİYORUM!" << std::endl;
    }
    void dersCalis() {
        std::cout << "Ders Çalışıyorum..." << std::endl;
    }
};

Öğretmen için ayrı bir sınıf;
class Ogretmen {
private:
    unsigned yas;
    char cinsiyet;
    unsigned verdigiDersKodu;
public:
    Ogretmen() {
        yas=18u;
        cinsiyet='E';
        verdigiDersKodu=0u;
    }
    void setYas(unsigned yeniYas) {
        if (yeniYas <18)
            std::cout << "Yaş 18 den Küçük Olamaz." << std::endl;
        else
            yas=yeniYas;
    }
    unsigned getYas() {
        return yas;
    }
    void setCinsiyet(char yeniCinsiyet) {
        if (yeniCinsiyet == 'E' || yeniCinsiyet == 'e' || yeniCinsiyet == 'K' || yeniCinsiyet ==
            ↪ 'k' || yeniCinsiyet == 'B' || yeniCinsiyet == 'b')
            cinsiyet=yeniCinsiyet;
        else
            std::cout << "Cinsiyet E,e,K,k,B,b Dışında Bir Değer Olamaz." << std::endl;
    }
    char getCinsiyet() {
        return cinsiyet;
    }
    void yasiniSoyle() {
        std::cout << "Yaşım:" << yas << std::endl;
    }
    void cinsiyetSoyle() {
        if (cinsiyet=='E' || cinsiyet=='e')
            std::cout << "Cinsiyetim: ERKEK" << std::endl;
        else if (cinsiyet=='K' || cinsiyet=='k')
            std::cout << "Cinsiyetim: KADIN" << std::endl;
        else
            std::cout << "Cinsiyetim: SÖYLEMEK İSTEMİYORUM!" << std::endl;
    }
}

```

```

    void dersVer() {
        std::cout << "Ders Veriyorum..." << std::endl;
    }
};

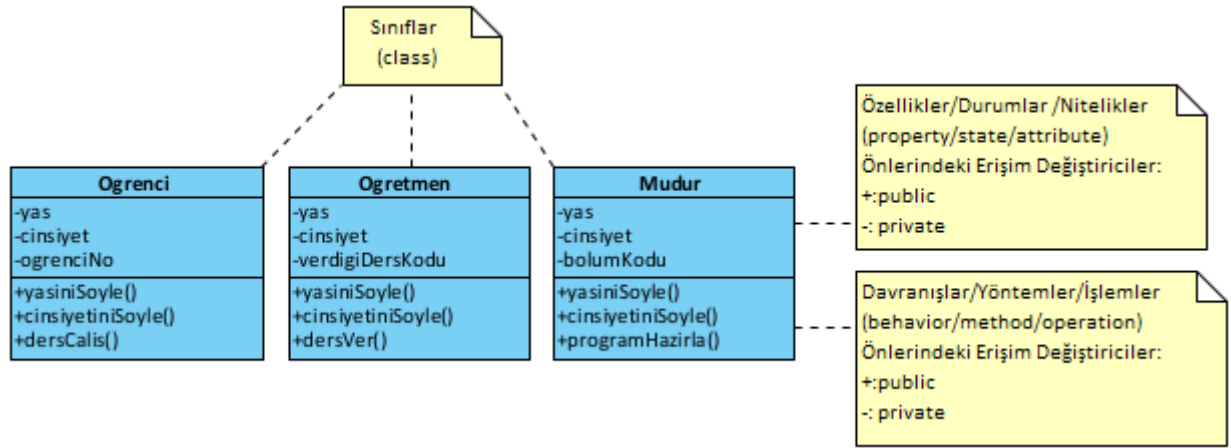
Müdür için ayrı bir sınıf;
class Mudur {
private:
    unsigned yas;
    char cinsiyet;
    unsigned muduruOlduguBolumKodu;
public:
    Mudur() {
        yas=18u;
        cinsiyet='E';
        muduruOlduguBolumKodu=0u;
    }
    void setYas(unsigned yeniYas) {
        if (yeniYas <18)
            std::cout << "Yaş 18 den Küçük Olamaz." << std::endl;
        else
            yas=yeniYas;
    }
    unsigned getYas() {
        return yas;
    }
    void setCinsiyet(char yeniCinsiyet) {
        if (yeniCinsiyet == 'E' || yeniCinsiyet == 'e' || yeniCinsiyet == 'K' || yeniCinsiyet == 'k' || yeniCinsiyet == 'B' || yeniCinsiyet == 'b')
            cinsiyet=yeniCinsiyet;
        else
            std::cout << "Cinsiyet E,e,K,k,B,b Dışında Bir Değer Olamaz." << std::endl;
    }
    char getCinsiyet() {
        return cinsiyet;
    }
    void yasiniSoyle() {
        std::cout << "Yaşım:" << yas << std::endl;
    }
    void cinsiyetSoyle() {
        if (cinsiyet=='E' || cinsiyet=='e')
            std::cout << "Cinsiyetim: ERKEK" << std::endl;
        else if (cinsiyet=='K' || cinsiyet=='k')
            std::cout << "Cinsiyetim: KADIN" << std::endl;
        else
            std::cout << "Cinsiyetim: SÖYLEMEK İSTEMİYORUM!" << std::endl;
    }
    void dersProgramiHazırla() {
        std::cout << "Ders Programı Hazırlıyorum..." << std::endl;
    }
};

```

Bu sınıfların her birinde `yas` ve `cinsiyet` gibi ortak özellikler (**property**) veya her sınıfta aynı kodu içerecek şekilde `yasiniSoyle()` ve `cinsiyetSoyle()` gibi ortak davranışlar (**behavior**) bulunur. Her sınıf için bunlar tekrar tekrar tanımlanırlar. Bunun gibi ortak özellikler ve davranışlar tek bir sınıf altında toplanabilir. Böylece proje zamanı kısılır ve daha yönetilebilir bir kod elde edilir.

Yazılım analizi yapılırken takım içerisinde iletişimi kolaylaştırmak için aşağıdaki **sınıf diyagramları** (**class diagram**) kullanılır. Bunlar UML (**Unified Modelling Language**) dilinin bir parçasıdır. Sonraki bölümde anlatılacaktır. Aşağıda yukarıda sınıfları kodlanan okul projesinin sınıf diyagramları verilmiştir;

Ortak özellik ve davranışların bir sınıfta toplanarak bu sınıftan miras alınmasına **kalıtım** (**inheritance**) adı verilir. Yukarıdaki örnekte ortak özellik ve davranışlar `Kisi` sınıfına toplanarak aşağıdaki şekilde kod yeniden düzenlenebilir.



Şekil 11.1: Okul Projesi Sınıfları UML Diyagramı

```

class Kisi {
private:
    unsigned yas;
    char cinsiyet;
public:
    Kisi() {
        yas=18u;
        cinsiyet='E';
    }
    void setYas(unsigned yeniYas) {
        if (yeniYas <18)
            std::cout << "Yaş 18 den Küçük Olamaz." << std::endl;
        else
            yas=yeniYas;
    }
    unsigned getYas() {
        return yas;
    }
    void setCinsiyet(char yeniCinsiyet) {
        if (yeniCinsiyet == 'E' || yeniCinsiyet == 'e' || yeniCinsiyet == 'K' || yeniCinsiyet == 'k' || yeniCinsiyet == 'B' || yeniCinsiyet == 'b')
            cinsiyet=yeniCinsiyet;
        else
            std::cout << "Cinsiyet E,e,K,k,B,b Dışında Bir Değer Olamaz." << std::endl;
    }
    char getCinsiyet() {
        return cinsiyet;
    }
    void yasiniSoyle() {
        cout << "Yaşım:" << yas << endl;
    }
    void cinsiyetSoyle() {
        if (cinsiyet=='E' || cinsiyet=='e')
            std::cout << "Cinsiyetim: ERKEK" << std::endl;
        else if (cinsiyet=='K' || cinsiyet=='k')
            std::cout << "Cinsiyetim: KADIN" << std::endl;
        else
            std::cout << "Cinsiyetim: SÖYLEMEK İSTEMİYORUM!" << std::endl;
    }
};
  
```

Yukarıda verilen ve **Kisi** olarak tanımlanan **genel sınıf** (general class), **taban sınıf** (base class) veya **ebeveyn sınıf** (parent class) olarak adlandırılır. Daha sonra tanımlanacak **Öğrenci**, **Öğretmen** ve **Müdür** sınıflarının ortak özellik ve davranışlarını barındırır.

Aşağıda **Kisi** taban sınıfından türemiş (derived) **Öğrenci**, **Öğretmen** ve **Müdür** sınıfları verilmiştir. Bu

sınıflar genel sınıf olan Kişi sınıfının yanında kendine özel özellik ve davranışlara da sahiptir. **Özel sınıf (private class)**, **türetilmiş sınıf (derived class)** veya **çocuk sınıf (child class)** olarak adlandırılan bu sınıflar, mirasçı olup, kendilerinden imal edilmiş nesneler; hem taban sınıfın özellik ve davranışlarını, hem de kendi özellik ve davranışlarını sergilerler.

```
class Ogrenci: public Kisi { //Kisi sınıfından türetilmiş Ogrenci sınıf
private:
    unsigned ogrenciNo;
public:
    Ogrenci() {
        ogrenciNo=0u;
    }
    void dersCalis() {
        std::cout << "Ders Çalışıyorum..." << std::endl;
    }
};

class Ogretmen:public Kisi { //Kisi sınıfından türetilmiş Ogretmen sınıf
private:
    unsigned verdigiDersKodu;
public:
    Ogretmen() {
        verdigiDersKodu=0u;
    }
    void dersVer() {
        std::cout << "Ders Veriyorum..." << std::endl;
    }
};

class Mudur:public Kisi { //Kisi sınıfından türetilmiş Mudur sınıf
private:
    unsigned muduruOlduguBolumKodu;
public:
    Mudur() {
        muduruOlduguBolumKodu=0u;
    }
    void dersProgramiHazirla() {
        std::cout << "Ders Programı Hazırlıyorum..." << std::endl;
    }
};
```

Programlamanın tamamlandığı ana fonksiyonda bu üç sınıftan da nesne imal edilmiş ve çeşitli davranışlar göstermesi için kendilerine ileti gönderilmiştir (**message-passing**).

```
#include <iostream>
//Sınıflar Buraya gelecek...
int main() {
    Ogrenci ilhan;
    Ogretmen mehmet;
    Mudur abdullah;

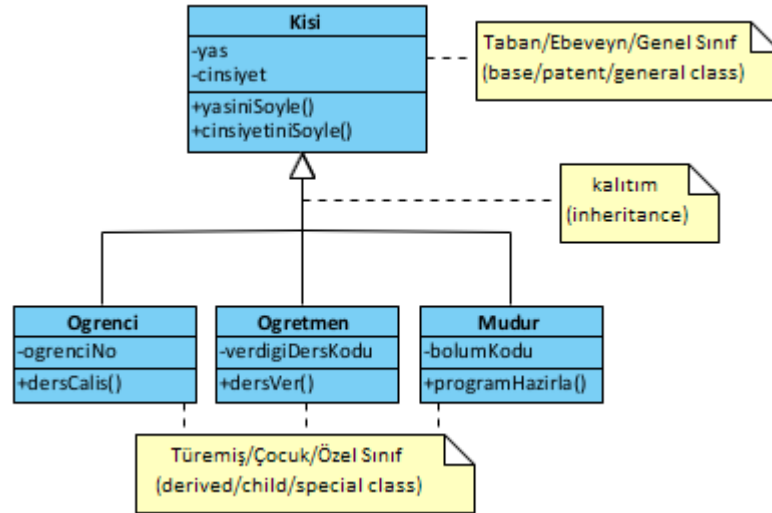
    ilhan.yasiniSoyle();
    ilhan.cinsiyetSoyle();
    ilhan.dersCalis();

    mehmet.yasiniSoyle();
    mehmet.cinsiyetSoyle();
    mehmet.dersVer();

    abdullah.yasiniSoyle();
    abdullah.cinsiyetSoyle();
    abdullah.dersProgramiHazirla();
}
```

Kalıtım uygulanmış haliyle sınıf diyagramları aşağıda verilmiştir;

Örneğimizden gidecek olursak, **Kisi** sınıfında yapılacak bir özellik veya davranış eklemesi anında bu



Şekil 11.2: Kalıtım Uygulanmış Okul Projesi Sınıfları UML Diyagramı

sınıftan türetilmiş sınıflarda yapılmış olur. Türetilmiş sınıflarda bununla ilgili değişiklik gerekmez.

Kalıtımın Üstünlükleri

Değişim yönetimini kolayca yapma üstünlüğünü veren kalıtımın (**inheritance**) iki önemli özelliği vardır;

1. Paylaşılan bir ortak kod (**shared code**) vardır.
2. Paylaşılan kodda yapılan değişiklik anında alt sınıflara yansır (**snap change**).

Türetilmiş sınıflar, taban sınıftan miras alırken üç farklı şekilde alabilirler;

1. **Umuma açık (public) kalıtım**: Taban sınıfın umuma açık üyelerini türetilmiş sınıfta umuma açık hale getirir ve taban sınıfın korumalı (protected) üyeleri de türetilmiş sınıfta korumalı olarak kalır.
2. **Mahrem (private) kalıtım**: Taban sınıfın umuma açık ve korumalı üyelerini türetilmiş sınıfta mahrem hale getirir.
3. **Korumalı (protected) kalıtım**: Taban sınıfın umuma açık ve korumalı üyelerini türetilmiş sınıfta korumalı hale getirir.

Aşağıda bu durumu açıklayan sınıf örneği verilmiştir;

```

class Base { //taban sınıf
public:
    /* Burada tanımlanan üyeler umuma açıktır. Yani bu üyelere sınıf dışı ve içinden
    → erişilebilir. */
    int x;
protected:
    /* burada tanımlı üyelere türeyen sınıftan türetilmiş sınıflardan erişilir. Bir başka deyişle
    → miras alan sınıflar tarafından burada tanımlanan üyelere erişilir. */
    int y;
private:
    /* burada tanımlı üyelere yalnızca bu sınıftan imal edilen nesneler erişir. Bir başka deyişle
    → yalnızca bu sınıf erişebilir.*/
    int z;
};

class PublicDerived: public Base { //public inheritance
    // Bu sınıfta x alanı(field), public olmuştur.
    // Bu sınıfta y alanı(field), protected olmuştur.
    // Bu sınıfta z alanı(field) ERİŞİLEMEZ olmuştur. -> Public-Derived
};

class PrivateDerived: private Base { //private inheritance
    // Bu sınıfta x alanı(field), private olmuştur.
    // Bu sınıfta y alanı(field), private olmuştur.
    // Bu sınıfta z alanı(field) ERİŞİLEMEZ olmuştur. -> Private-Derived
};
  
```

```

};
class ProtectedDerived: protected Base { //protected inheritance
    // Bu sınıfta x alanı(field), protected olmuştur.
    // Bu sınıfta y alanı(field), protected olmuştur.
    // Bu sınıfta z alanı(field) ERIŞİLEMEZ olmuştur. -> Protected-Derived
};

```

Kalıtımda üyelere erişim, özet olarak aşağıdaki tabloda verilmiştir; Korunmalı (**protected**) olarak taban sınıftan

Üyelere Erişim	public	protected	private
Aynı Sınıf İçinde	var	var	var
Türemiş Sınıfta	var	var	yok
Sınıf Dışından	var	yok	yok

Tablo 11.1: Kalıtımda Sınıf Üyelerine Erişim

alınan durum ve davranışlara ait bu durumu gösteren bir örnek aşağıda verilmiştir;

```

#include <iostream>
class Kisi {
protected: //Burada tanımlana alan ve davranışlara türemiş sınıflardan da erişilebilir.
    unsigned yas;
    char cinsiyet;
public:
    Kisi() {
        yas=18;
        cinsiyet='E';
    }
    void setYas(unsigned yeniYas) {
        if (yeniYas <18)
            std::cout << "Yaş 18 den Küçük Olamaz." << std::endl;
        else
            yas=yeniYas;
    }
    unsigned getYas() {
        return yas;
    }
    void setCinsiyet(char yeniCinsiyet) {
        if (yeniCinsiyet == 'E' || yeniCinsiyet == 'e' || yeniCinsiyet == 'K' || yeniCinsiyet == 'k' || yeniCinsiyet == 'B' || yeniCinsiyet == 'b')
            cinsiyet=yeniCinsiyet;
        else
            std::cout << "Cinsiyet E,e,K,k,B,b Dışında Bir Değer Olamaz." << std::endl;
    }
    char getCinsiyet() {
        return cinsiyet;
    }
    void yasiniSoyle() {
        std::cout << "Yaşım:" << yas << std::endl;
    }
    void cinsiyetSoyle() {
        if (cinsiyet=='E' || cinsiyet=='e')
            std::cout << "Cinsiyetim: ERKEK" << std::endl;
        else if (cinsiyet=='K' || cinsiyet=='k')
            std::cout << "Cinsiyetim: KADIN" << std::endl;
        else
            std::cout << "Cinsiyetim: SÖYLEMEK İSTEMİYORUM!" << std::endl;
    }
};

class Ogrenci: public Kisi {
private:
    unsigned ogrenciNo;
public:
    Ogrenci() {

```

```

        yas=35u; // Artık yas alanına bu türemiş sınıftan da erişilebiliyor.
        cinsiyet='K'; // Artık cinsiyet alanına bu türemiş sınıftan da erişilebiliyor.
        ogrenciNo=0u;
    }
    void dersCalis() {
        std::cout << "Ders Çalışıyorum..." << std::endl;
    }
};
int main() {
    Ogresci ayse;
    ayse.yasiniSoyle();
    ayse.cinsiyetSoyle();
    ayse.dersCalis();
}

```

Çoklu Kalıtım

C++ dilinde bir türemiş sınıfın birden fazla ebeveyn sınıftan miras alması istenmez. Mantık olarak bir ev, birden fazla mimari plandan hazırlanmayacağından, **çoklu kalıtım (multiple inheritance)** sınıf tasarımında kullanılmaz!

Ancak C++ dilinde 11.19 başlığında inceleyeceğimiz **arayüzler (interface)** de sınıf olarak kodlandığından sanki çoklu kalıtım varmış gibi algılanmaktadır.

Saf nesne yönelimli diller olan Java ve .Net dillerinde arayüzler **interface** anahtar kelimesiyle tanımlandığından **birden çok sınıftan miras alma engellenmiştir**.

Türetilmiş bir sınıf, aşağıdaki istisnalar dışında tüm taban sınıf yöntemlerini miras alır;

- ◊ Taban sınıfın yapıcıları, **yıkıcıları (destructor)** ve **kopya yapıcıları (copy constructor)**.
- ◊ Taban sınıfın aşırı yüklenmiş işleçleri.
- ◊ Taban sınıfın **arkadaş (friend)** fonksiyonları.

Bu kavramlar ilerleyen bölümlerde anlatılacaktır.

11.10 Yöntemlerin Aşırı Yüklenmesi

Fonksiyonlar için 7.12 başlığında anlatılan aşırı yükleme, sınıflardaki **yöntemlere (method)** de uygulanabilir. Yani farklı parametrelerle yöntem yeniden tanımlanabilir. Aşırı yüklemede yöntemin kimliği ve geri dönüş tipi değişmez. Farklı argümanlar ile yöntemin gövdesi yeniden tanımlanır. Yani aynı davranışın, farklı yöntemlerle yeniden tanımlanmasıdır.

```

#include <iostream>
class Karmasik {
public:
    float gercek, sanal;
    Karmasik() { // varsayılan yapıcı (default constructor)
        gercek = 0.0;
        sanal = 0.0;
    }
    void yaz() { //yaz yöntemi
        std::cout<< gercek << "+" << sanal << "+"i" << std::endl;
    }
    void yaz(char pKarakter) { // aşırı yüklenmiş yaz yöntemi
        //aşırı yüklenmiş yaz yöntemi
        std::cout << pKarakter << gercek << "+" << sanal << "+"i" <<pKarakter << std::endl;
    }
    void yaz(char pIlkKarakter, char pSonKarakter) { //aşırı yüklenmiş bir başka yaz yöntemi
        std::cout << pIlkKarakter << gercek << "+" << sanal << "i" << pSonKarakter << std::endl;
    }
};
int main(void) {
    Karmasik c1;
    c1.gercek=1;
    c1.sanal=2;
    c1.yaz(); // 1+2i
}

```

```

c1.yaz('@');           // @1+2i@
c1.yaz('[', ']');      // [1+2i]
}

```

Hali hazırda tanımlı olan **işleçlere** (**operator**) de bizim tanımladığımız sınıflarda aşırı yükleme yapılabilir. Ya da istenmeyen işleçler **=delete** belirleyicisi (**=delete specifier**) ile sınıfla işlem yapması kaldırılabilir. Bu işleç aynı zamanda kalıttan davranılan davranışları ve yapıları kaldırmak için de kullanılabilir.

Aşağıdaki örnekte toplama işlecine **aşırı yükleme** (**operator overloading**) yapılmış ve çıkarma işleci **kaldırılmıştır**. Ayrıca nesneleri atama işleciyle kopyalayan **kopya yapıcı** (**copy constructor**) da yazılmıştır.

```

#include <iostream>
class Karmasik {
public:
    float gercek, sanal;
    Karmasik() { // varsayılan yapıcı (default constructor)
        gercek = 0.0f;
        sanal = 0.0f;
    }
    Karmasik(float pGercek, float pSanal): gercek(pGercek), sanal(pSanal) {
    }
    Karmasik(float pGercek): gercek(pGercek) {
        sanal = 0.0f;
    }
    void yaz() { //yaz yöntemi
        std::cout<< gercek << "+" << sanal << "+"i" << std::endl;
    }
    Karmasik operator+(const Karmasik& pKarmasik) { /* + işleci aşırı yükleniyor (overloading):
        ↪ iki karmaşık sayı toplanıyor */
        std::cout << "+ işleci karmaşık sayı parametresiyle çalışacak." << std::endl;
        Karmasik toplam;
        toplam.gercek = gercek + pKarmasik.gercek;
        toplam.sanal = sanal + pKarmasik.sanal;
        return toplam;
    }
    Karmasik operator+(const float pGercek) { /* + işleci aşırı yükleniyor (overloading):
        ↪ karmaşık sayı ile gerçek sayı toplanıyor */
        std::cout << "+ işleci gerçek sayı parametresiyle çalışacak." << std::endl;
        Karmasik toplam;
        toplam.gercek = gercek + pGercek;
        toplam.sanal = sanal;
        return toplam;
    }
    Karmasik(const Karmasik& pKarmasik) { /* Aşırı yüklenmiş kopya yapıcı (copy constructor):
        ↪ atama (=) için tanımlanmalıdır. */
        std::cout << "Kopya Yapıcı Çalıştı." << std::endl;
        gercek= pKarmasik.gercek;
        sanal=pKarmasik.sanal;
    }
    void operator-(const Karmasik &) = delete; // Çıkarma işlemi engellenmiş */
};

int main(void) {
    Karmasik karmasik1;
    karmasik1.yaz();
    Karmasik karmasik2(1.0f,1.0f);
    karmasik2.yaz();
    Karmasik karmasik3=karmasik2;
    karmasik3.yaz();
    Karmasik karmasik4=karmasik2+karmasik3;
    karmasik4.yaz();
    Karmasik karmasik5=karmasik2+11.0f;
    karmasik5.yaz();
}
/*Program Çıktısı:

```

```
0+0i
1+1i
Kopya Yapıcı Çalıştı.
1+1i
+ işleci karmaşık sayı parametresiyle çalışacak.
2+2i
+ işleci gerçek sayı parametresiyle çalışacak.
12+1i
*/
```

11.11 Yapıcıların Aşırı Yüklenmesi

Yapıcıların görevi kısaca imal edilecek nesnelerin durumlarına (**state**) ilk değer vermektir. Durumlara ilişkin veriler de alanlarda (**field**) tutulurlar. Nesneleri de dışarıdan verilen parametrelerle farklı durumlarda imal etmek için aşırı yüklenmiş yapıcıları kullanabiliriz. Yani farklı parametrelerle yapıcıları yeniden tanımlayabiliriz.

Yapıcıların aşırı yüklenmesinin çeşitli üstünlükleri vardır;

- ◊ Nesne imal etmede esneklik: Aynı sınıftan farklı durumlara sahip nesneler imal edilebilir.
- ◊ Gelişmiş kod bakımı ile daha temiz ve okunabilir kod oluşur.
- ◊ Nesne imal etme mantığı diğer nesnelerden/programlardan gizlenir.
- ◊ Kopya yapıcılar ile nesne klonlama basitleşir.

§11.11.1 Varsayılan Yapıcı

Hiç parametresi olmayan yapıcıya **varsayılan yapıcı** (**default constructor**) adı verilir.

```
#include <iostream>
class Karmasik {
public:
    float gercek, sanal;
    Karmasik() { /* varsayılan yapıcı (default constructor):
        Gerçek ve sanal kısım 0 olarak nesneler imal edilir. */
        gercek = 0.0;
        sanal = 0.0;
    }
    void yaz() { //yaz yöntemi
        std::cout<< gercek << "+" << sanal << "+"i" << std::endl;
    }
};
int main(void) {
    Karmasik c1; // varsayılan yapıcı ile imal edilen c1 nesnesi
    c1.yaz(); // 0+0i
}
```

§11.11.2 Parametrelili Yapıcılar

Varsayılan yapıcı dışındaki aşırı yüklenmiş yapıcılar ise **parametrelili yapıcı** (**parameterized constructor**) adını alır.

```
#include <iostream>
class Karmasik {
public:
    float gercek, sanal;

    Karmasik() { /* Varsayılan Yapıcı (default constructor): Gerçek ve sanal kısım 0 olarak
        ↪ nesneler imal edilir. */
        gercek = 0.0;
        sanal = 0.0;
    }
    Karmasik(float pGercek, float pSanal): gercek(pGercek), sanal(pSanal) { /* Aşırı yüklenmiş
        ↪ parametrelili yapıcı (parameterized constructor)*/
    }
    /*
    //Yukarıdaki yapıcı bu şekilde de yazılabilir;
    Karmasik(float pGercek, float pSanal) {
```

```

        gercek = pGercek;
        sanal = pSanal;
    }
    /*
    Karmasik(float pGercek): gercek(pGercek), sanal(0.0f){ /* Aşırı yüklenmiş bir başka
    ↪ parametrelili yapıcı (parameterized constructor) */
    }
    /*
    //Yukarıdaki yapıcı bu şekilde de yazılabilir;
    Karmasik(float pGercek) {
        gercek=pGercek;
        sanal=0.0f;
    }
    */
    void yaz() { //yaz yöntemi
        std::cout<< gercek << "+" << sanal << "+i" << std::endl;
    }
};
int main(void) {
    Karmasik c1; // varsayılan yapıcı ile imal edilen c1 nesnesi
    c1.yaz(); // 0+0i
    Karmasik c2(2.0,1.0); // aşırı yüklenmiş yapıcı ile imal edilen c2 nesnesi
    c2.yaz(); // 2+1i
    Karmasik c3(12.0);
    c3.yaz(); //12+0i
}

```

§11.11.3 Devredilen Yapıcı

Üstte örneği verilen ikinci yapıcı, **devredilen yapıcı** (**delegating constructor**) ile aşağıdaki şekilde de tanımlanabilir;

```

#include <iostream>
class Karmasik {
public:
    float gercek, sanal;
    Karmasik() { /* varsayılan (default constructor): Gerçek ve sanal kısım 0 olarak nesneler
    ↪ imal edilir. */
        gercek = 0.0f;
        sanal = 0.0f;
    }
    Karmasik(float pGercek, float pSanal): gercek(pGercek), sanal(pSanal) {
    }
    Karmasik(float pGercek): Karmasik (pGercek, 0.0f) { //Devredilen Yapıcı (delegating
    ↪ constructor)
    }
    void yaz() { //yaz yöntemi
        std::cout<< gercek << "+" << sanal << "+i" << std::endl;
    }
};
int main(void) {
    Karmasik c1; // varsayılan yapıcı ile imal edilen c1 nesnesi
    c1.yaz(); // 0+0i
    Karmasik c2(2.0,1.0); // aşırı yüklenmiş yapıcı ile imal edilen c2 nesnesi
    c2.yaz(); // 2+1i
    Karmasik c3(12.0);
    c3.yaz(); //12+0i
}

```

§11.11.4 Kopya Yapıcı

Aynı sınıftan bir nesneyi bir başka imal edilmiş nesnenin durumlarına sahip olarak imal edebiliriz. Bir nesneyi bir yapıcıya argüman olarak geçirmek ve yeni nesneye argümanın durumlarını vermek gerekir. Bu yapıcıya **kopya yapıcı** (**copy constructor**) adı verilir;


```
#include <iostream>
class Karmasik {
public:
    float gercek, sanal;
    Karmasik() { /* varsayılan yapıcı (default constructor)*/
        gercek = 0.0;
        sanal = 0.0;
    }
    Karmasik(float pGercek, float pSanal): gercek(pGercek), sanal(pSanal) { /* parametrelili yapıcı
    → (parameterized constructor)*/
    }
    Karmasik(Karmasik& pKarmasik) { /* Kopya yapıcı (copy constructor): Parametre olarak bir
    → başka karmaşık sayı veriliyor ve Gerçek ve sanal durumları parametreden gelen karmaşık
    → sayı ile aynı olacak şekilde nesneler imal edilir. */
        gercek= pKarmasik.gercek;
        sanal=pKarmasik.sanal;
    }
    void yaz() { //yaz yöntemi
        std::cout<< gercek << "+" << sanal << "+"i" << std::endl;
    }
};

int main(void) {
    Karmasik c1; // öntanımlı yapıcı ile imal edilen c1 nesnesi
    c1.yaz();    // 0+0i
    Karmasik c2(2.0,1.0); // parametrelili yapıcı ile imal edilen c2 nesnesi
    c2.yaz();    // 2+1i
    Karmasik c3(c2); // kopya yapıcı ile imal edilen c3 nesnesi
    c3.yaz();    // 2+1i
}
```

§11.11.5 Atama İşlecinin Aşırı Yüklenmesi

Kopya yapıcı gibi çalışan ancak mevcut iki nesne üzerinde işlem yapan atama işlecine ilişkin aşırı yüklemeye bazen **eşitlik yapıcısı** (**assignment constructor**) adı verilir. **Aslında bu tam bir yapıcı değildir** mevcut nesnenin durumunu bir başka nesnenin durumuna eşler.

```
#include <iostream>
class Karmasik {
public:
    float gercek, sanal;
    Karmasik() { /* öntanımlı yapıcı (default constructor)*/
        gercek = 0.0;
        sanal = 0.0;
    }
    Karmasik(float pGercek, float pSanal): gercek(pGercek), sanal(pSanal) {
    }
    Karmasik(Karmasik& pKarmasik) { /* Kopya yapıcı (copy constructor)*/
        this->gercek= pKarmasik.gercek;
        this->sanal=pKarmasik.sanal;
    }
    Karmasik& operator=(const Karmasik& pKarmasik) { /* atama işleci aşırı yüklemesi */
        this->gercek= pKarmasik.gercek;
        this->sanal=pKarmasik.sanal;
        return *this;
    }
    void yaz() { //yaz yöntemi
        std::cout<< gercek << "+" << sanal << "+"i" << std::endl;
    }
};

int main(void) {
    Karmasik c1; // öntanımlı yapıcı ile imal edilen c1 nesnesi
    c1.yaz();    // 0+0i
    Karmasik c2(2.0,1.0); // parametrelili yapıcı ile imal edilen c2 nesnesi
```

```

c2.yaz();    // 2+1i
Karmasik c3(c2); // kopya yapıcı ile imal edilen c3 nesnesi
c3.yaz();    // 2+1i
c3=c1;       // atama işleci aşırı yüklemesi
c3.yaz();    // 0+0i
}

```

§11.11.6 Varsayılan Yapıcının Değiştirilmesi

Yukarıda açıklanan **varsayılan yapıcı** (**default constructor**), **=default** belirleyicisi (**=default specifier**) ile değiştirilebilir. Bu belirleyici aşırı yüklenmiş fonksiyonlarda da kullanılabilir.

C++ standartlarında, **float** gibi temel tipleri parametre alan bir yapıcıyı doğrudan **= default**; şeklinde tanımlayamazsınız! Derleyici bu parametreleri nesne üyelerine nasıl eşleyeceğini bilemez. Ancak üyeleri sınıf içinde ilk olarak hangisinin seçileceğini belirleyebilirsiniz. Parametresiz yapıcıyı **default** yaparak aynı etkiyi çok daha temiz bir şekilde elde edebilirsiniz.

Aşağıda bu belirleyiciye ilişkin örnek verilmiştir;

```

#include <iostream>
class Karmasik {
public:
    // Üyeleri burada başlatıyoruz (C++11 ve sonrası için en iyi pratik)
    float gercek = 0.0f;
    float sanal = 0.0f;
    // Öntanımlı yapıcıyı 'default' yaparak yukarıdaki değerleri kullanmasını sağlıyoruz
    Karmasik() = default;
    // Parametrelili yapıcı: İlklendirme listesi ile en verimli yol
    Karmasik(float pGercek, float pSanal) : gercek(pGercek), sanal(pSanal) {}
    // Kopya yapıcıyı da derleyiciye bırakmak (default) en performanslısıdır
    Karmasik(const Karmasik&) = default;
    // Atama operatörünü de derleyiciye bırakabiliriz
    Karmasik& operator=(const Karmasik&) = default;
    void yaz() const {
        std::cout << gercek << " + " << sanal << "i" << std::endl;
    }
};

int main() {
    Karmasik k1;
    k1.yaz(); //0 + 0i
    k1.gercek=3;
    k1.sanal=5;
    Karmasik k2=k1;
    k2.yaz(); //3 + 5i
}

```

Kalıtım (**inheritance**) durumunda ise taban sınıfın varsayılan yapıcısı çağrılır. Bu duruma aşağıdaki örnek verilebilir;

```

#include <iostream>
class Kisi {
private:
    unsigned yas;
    char cinsiyet;
public:
    Kisi() { // Taban Sınıf Varsayılan Yapıcısı
        yas = 18;
        cinsiyet = 'E';
        std::cout << "-> Kisi varsayılan yapicisi calisti (Yas: 18, Cinsiyet: E)" << std::endl;
    }
    // Bilgi veren metodlar
    void yasiniSoyle() { std::cout << "Yasim: " << yas << std::endl; }
    void cinsiyetSoyle() {
        std::cout << "Cinsiyetim: " << (cinsiyet == 'E' ? "ERKEK" : "KADIN") << std::endl;
    }
};

```

```

class Ogresci : public Kisi {
private:
    unsigned ogresciNo;
public:
    Ogresci() { // Türetilmiş Sınıf Varsayılan Yapıcısı
        ogresciNo = 0;
        // Burada Kisi() gizli olarak (implicit) zaten çağırıldı.
        std::cout << "-> Ogresci varsayılan yapicisi calisti (No: 0)" << std::endl;
    }
    void dersCalis() {
        std::cout << "Ogresci ders calisiyor..." << std::endl;
    }
};

int main() {
    std::cout << "Ogresci nesnesi olusturuluyor..." << std::endl;
    // Ogresci nesnesi olustugu an silsile baslar:
    // 1. Kisi() cagrilir
    // 2. Ogresci() cagrilir
    Ogresci ilhan;
    ilhan.yasiniSoyle(); // Kisi'den miras alindi
    ilhan.cinsiyetSoyle(); // Kisi'den miras alindi
    ilhan.dersCalis(); // Ogresci'ye ozgu
}

```

§11.11.7 Taşıma Yapıcısı

Bir kopya yapıcı (**copy constructor**), mevcut bir nesnenin durumunu başka bir nesneye aktararak yeni bir örnek oluşturmak için kullanılır. Bu işlem için genellikle aşağıdaki söz dizimi tercih edilir:

```

// Kopya Yapıcı
Karmasik(const Karmasik &diger) {
    this->gercek = diger.gercek;
    this->sanal = diger.sanal;
}

```

Burada **Karmasik** sınıfı, aynı türden başka bir nesneye sabit (**const**) bir referans olarak üyeleri elle kopyalar. Eğer özel bir kopyalama mantığı gerekmiyorsa, tüm üyelerin otomatik kopyalanması için **Karmasik(const Karmasik&) = default;** ifadesi de kullanılabilir.

Öte yandan, modern C++ ile gelen taşıma yapıcısı (**move constructor**), bir nesneyi kopyalamak yerine durumunu (veya kaynaklarını) yeni nesneye taşımak için kullanılır. Bu işlemde **lvalue** referansı yerine **rvalue** referansı (**rvalue reference**) (**&&**) kullanılır. Taşıma semantiği, özellikle dinamik bellek yönetimi içeren sınıflarda performans açısından "sahipliğin aktarılmasına" veya "durumun devralınmasına" izin verir.

```

// Taşıma Yapıcısı
Karmasik(Karmasik &&diger) noexcept {
    this->gercek = diger.gercek;
    this->sanal = diger.sanal;
    // Kaynak nesnenin durumunu sıfırlıyoruz
    diger.gercek = 0.0f;
    diger.sanal = 0.0f;
}

```

Burada, durumu taşınan (kaynak) nesnenin değerlerinin sıfırlandığına dikkat edilmelidir. Taşıma işlemi, orijinal örnekten **durum çalmaya** (**pilfering**) olanak tanır. Orijinal örneğin bu işlemden sonraki hali, programın tutarlılığı açısından kritik bir öneme sahiptir. Eğer değerleri sıfırlamasaydık, taşıma sonrası her iki nesne de aynı veriye sahip olurdu ki bu, özellikle gösterici (**pointer**) içeren yapılarda bellek hatalarına yol açabilirdi.

Aşağıdaki kullanımda bu fark açıkça görülmektedir:

```

Karmasik sayi1;
sayi1.gercek = 5.5f;
sayi1.sanal = 2.0f;
// sayi1'in durumu sayi2'ye taşınır
Karmasik sayi2(std::move(sayi1));
std::cout << "Sayi 1: " << sayi1.gercek << " + " << sayi1.sanal << "i" << std::endl; // 0 + 0i

```

```
std::cout << "Sayı 2: " << sayi2.gercek << " + " << sayi2.sanal << "i" << std::endl; // 5.5 +
↳ 2i
```

Böylece, bellek üzerinde yeni bir kopyalama maliyetine katlanmadan, eski bir nesnenin sahip olduğu veriyi yeni bir nesneye verimli bir şekilde aktarmış oluruz.

§11.11.8 Yapıcılarda Varsayılan Argümanlar

Fonksiyonlarda 7.11 başlığında işlenen **varsayılan argümanlar** (**default argument**) yapıcılarda da kullanılabilir. Karmaşık sayılar örneği aşağıda verilmiştir;

```
#include <iostream>
class Karmasik {
    float gercek, sanal; //bu üyeler varsayılan olarak private
public:
    Karmasik(float pGercek=0.0f, float pSanal=0.0f): gercek(pGercek), sanal(pSanal) { }
    // Varsayılan argümanlara yapıcı tanımı:
    void yaz() { //yaz yöntemi
        std::cout<< gercek << "+" << sanal << "i" << std::endl;
    }
    Karmasik operator+(const Karmasik& pKarmasik) {
        Karmasik toplam;
        toplam.gercek = gercek + pKarmasik.gercek;
        toplam.sanal = sanal + pKarmasik.sanal;
        return toplam;
    }
    Karmasik operator+(const float pGercek) {
        Karmasik toplam;
        toplam.gercek = gercek + pGercek;
        toplam.sanal = sanal;
        return toplam;
    }
    Karmasik(const Karmasik& pKarmasik) { // copy constructor
        gercek= pKarmasik.gercek;
        sanal=pKarmasik.sanal;
    }
};

int main(void) {
    Karmasik c1; // Varsayılan argümanlara Karmasik(0.0f,0.0f) yapıcısı kullanılarak imal edildi.
    c1.yaz();
    Karmasik c2(1.0,1.0); // Karmasik(1.0f,1.0f) yapıcısı ile imal edildi.
    c2.yaz();
    Karmasik c3(1.0); // Varsayılan argümanlara Karmasik(1.0f,0.0f) yapıcısı kullanılarak imal
    ↳ edildi.
    c3.yaz();
    Karmasik c4=c2+c3; // c4 kopya yapıcısı ile imal edildi.
    c4.yaz();
    c4=c2+c3; // c4 kopya yapıcısı ile imal edildi.
    c4.yaz();
}
```

11.12 using Anahtar Kelimesi ile Yapıcıların Miras Alınması

using anahtar kelimesi, 10.11.4 başlığındaki kullanımı yanında, C++11/14 ile **using** anahtar kelimesi, türetilmiş bir sınıfın (**derived class**), temel sınıfın (**base class**) tüm yapıcıları tek satırda miras almasını sağlar.

```
class Temel {
public:
    Temel(int x) { /*...*/ }
    Temel(double y, int z) { /*...*/ }
};

class Turemis : public Temel {
public:
    using Temel::Temel; // Temel sınıfın TÜM yapıcılarını içeri aktarır!
};
```

```
int main() {
    // Artık Turemis sınıfı için int veya double alan yapıcılarını tek tek yazmaya gerek yok.
    Turemis t(10); //türemiş sınıfta using kullanıldığı için bu yapılabilir!
}
```

11.13 Nesne Yıkıcı

Yıkıcı (destructor), bir nesne yok edileceği zaman otomatik olarak çağrılan bir yöntemdir. Bir sınıf içinde yalnızca bir yıkıcı tanımlanabilir. Yıkıcı, bir nesne yok edilmeden önce çağrılacak son yöntemdir. Aşağıdaki gibi tanımlanır.

```
~sınıf-kimliği() {
    // ...
}
```

Yıkıcılar, daha çok kaynak kullanan nesnelerin yok edilmesi öncesinde çağrılmak üzere kodlanır. Çalışma mantığına ilişkin örnek verilmiştir;

```
#include <iostream>

static int nesneSayaci=0;

class Karmasik {
    float gercek, sanal;
public:
    Karmasik(float pGercek=0.0f, float pSanal=0.0f): gercek(pGercek), sanal(pSanal) {
        nesneSayaci++;
        std::cout << nesneSayaci << " nesne imal edildi" << std::endl;
    }
    void yaz() { //yaz yöntemi
        std::cout<< gercek << "+" << sanal << "+i" << std::endl;
    }
    Karmasik operator+(const Karmasik& pKarmasik) {
        Karmasik toplam;
        toplam.gercek = gercek + pKarmasik.gercek;
        toplam.sanal = sanal + pKarmasik.sanal;
        return toplam;
    }
    Karmasik(const Karmasik& pKarmasik) {
        gercek= pKarmasik.gercek;
        sanal=pKarmasik.sanal;
    }
    ~Karmasik() {
        nesneSayaci--;
        std::cout << nesneSayaci << " nesne yok edildi" << std::endl;
    }
};

int main(void) {
    Karmasik c1;
    c1.yaz();
    Karmasik c2(3.0,5.0);
    c2.yaz();
    Karmasik c3=c1+c2;
    c3.yaz();
}

/*Program Çıktısı:
1 nesne imal edildi
0+0i
2 nesne imal edildi
3+5i
3 nesne imal edildi
3+5i
2 nesne yok edildi
1 nesne yok edildi
```

```
0 nesne yok edildi
*/
```

11.14 friend Anahtar Kelimesi

Bir sınıfın **arkadaş** (**friend**) yöntemi, o sınıfın bloğu dışında tanımlanır ancak sınıfın tüm mahrem (**private**) ve korumalı (**protected**) üyelerine erişim hakkına sahiptir. Arkadaş yöntemlerin bildirimleri/prototipleri sınıf içerisinde yer almasına rağmen, arkadaşlar üye fonksiyon değildir.

```
#include <iostream>
class Karmasik {
    float gercek, sanal; //private members
public:
    Karmasik(float pGercek=0.0, float pSanal=0.0) {
        this->gercek=pGercek;
        this->sanal=pSanal;
    }
    friend void yaz(Karmasik pKarmasik);
};
void yaz(Karmasik pKarmasik) {
    std::cout << pKarmasik.gercek << "+" <<pKarmasik.sanal <<"i" << std::endl;
}
int main(void) {
    Karmasik c1(1.0,2.0);
    yaz(c1); // 1+2i
    Karmasik c2(3.0,5.0);
    yaz(c2); // 3+5i
}
```

Buna, bir sınıfla bir akışının (**stream**) ilişkilendirilmesi örnek verilebilir. Nesne verilerinin akıştan çıkarılmasında **ayıklama işleci** (**extraction operator**) (**>>**) kullanılabilir. Benzer şekilde nesne verilerinin akışlara göndermek için **ekleme işleci** (**insertion operator**) (**<<**) kullanılabilir. Karmaşık sayı sınıfı için aşağıdaki gibi bir örnek verilebilir;

```
#include <iostream>
class Karmasik {
    float gercek, sanal; // Varsayılan olarak private
public:
    // Parametrelili yapıcı (Modern ilklendirme listesi kullanımı)
    Karmasik(float pGercek = 0.0f, float pSanal = 0.0f)
    : gercek(pGercek), sanal(pSanal) {}
    // Çıkış Akışı (Output Stream) İşleci
    friend std::ostream& operator<<(std::ostream& output, const Karmasik& pKarmasik) {
        output << pKarmasik.gercek << "+" << pKarmasik.sanal << "i";
        return output;
    }
    // Giriş Akışı (Input Stream) İşleci
    friend std::istream& operator>>(std::istream& input, Karmasik& pKarmasik) {
        // Not: Giriş operatöründe nesne const olamaz çünkü değerlerini değiştireceğiz.
        input >> pKarmasik.gercek >> pKarmasik.sanal;
        return input;
    }
};
int main(void) {
    Karmasik c1(1.0f, 2.0f);
    std::cout << "c1 nesnesi: " << c1 << std::endl; // 1+2i
    Karmasik c2;
    std::cout << "Lutfen gercek ve sanal kısmi giriniz (örn: 3.5 4.2): ";
    std::cin >> c2; // Kullanıcıdan veri alma
    std::cout << "Girilen c2 nesnesi: " << c2 << std::endl;
}
```

11.15 Statik Sınıf Üyeleri

C++ Dilinde **statik üye yöntemi** (**static member method**), imal edilecek herhangi bir nesneye değil, sınıfın kendisine ait olan özel bir fonksiyon türüdür. Bu yöntemleri tanımlamak için statik anahtar kelimesi kullanılır. Sınıfın bir örneği olacak nesne imal edilmesine gerek kalmadan, sadece sınıf kimliğini kullanarak doğrudan çağrılabilirler. Bu tür yöntemlere **yardımcı yöntem** (**utility method**) adı verilir. Bu yöntemler program çalışmaya başladığında var olur ve program bitinceye kadar erişilebilirler. Ayrıca yalnızca sınıfın statik üyeleri veya diğer statik fonksiyonlarıyla çalışabilirler. **static** bir üyeye sınıf içerisinde veya yapısında ilk değer veremeyiz! Bu nedenle sınıf dışında veri tipi belirterek başlatmalıyız!

```
#include <iostream>
class YardimciSinif {
public:
    static int genelSayac;
    static void programciAdi() {
        std::cout << "Programcı Adı: İlhan OZKAN" << std::endl;
        std::cout << genelSayac << std::endl;
    }
};
/* sınıf üyesini sınıf dışında veri tipi belirterek başlatma */
int YardimciSinif::genelSayac=100;
int main(void) {
    YardimciSinif::programciAdi(); /* static üye yöntemi çağırma */
    YardimciSinif::genelSayac--; /* static üye alana erişim */
    YardimciSinif::programciAdi();
}
```

Statik yöntemlere benzer şekilde statik alanlar da tanımlanabilir. Statik olan bu veriler veri bellekte (**data segment**) saklanırlar ve bu veri üyeleri de statik yöntemler gibi kullanılır. Statik üyeler genellikle bir sınıfın tüm örnekleri arasında paylaşılması gereken paylaşılan kaynakları veya sayaçları yönetmek için kullanılır. İleride anlatılacak olan **tasarım desenlerinden** (**design pattern**) **tekil nesne deseni** (**singleton pattern**) statik üyelere örnek olarak verilebilir. Bu desende sınıftan yalnızca bir nesne/örnek imal edilen ve bu tekil nesneye evrensel erişim sağlayan bir tasarım deseni. Burada statik üyeler, sınıfın tek bir paylaşılan örneğinin bakımına izin verdikleri için bu deseni uygulamak için mükemmeldir.

11.16 const Sınıf Üyeleri

Sınıfların **const yöntemleri** (**const method**), veri üyelerinin yani durumları (**state**) tutan alanları (**field**) değiştirme izni verilmeyen fonksiyonlardır. Bir üye fonksiyonunu **const** yapmak için, **const** anahtar kelimesi fonksiyon prototipine ve ayrıca fonksiyon tanımındaki başlığına eklenir.

Üye yöntemler gibi bir sınıfın nesneleri de **const** olarak bildirilebilir. **const** olarak bildirilen bir nesne değiştirilemez ve bu nedenle, bu fonksiyonlar nesneyi değiştirmemeyi garantilediğinden, yalnızca **const** üye yöntemlerini çağırabilir.

Bir **const** nesnesi, nesne bildirimine **const** anahtar sözcüğünü ön ek olarak ekleyerek tanımlanabilir. Bu nesneler **değiştirilemez** (**immutable**) olarak adlandırılır. **const** nesnelerinin veri üyesini değiştirmeye yönelik herhangi bir girişim, derleme zamanı hatasıyla (**compile-time error**) sonuçlanır.

```
#include <iostream>
class Sinif {
private:
    int alan;
public:
    Sinif() {
        alan=-1;
    }
    int getAlan() const {
        return alan;
    }
    void setAlan(int pAlan) { // Bu yöntem const tanımlanırsa hata alınır!
        alan=pAlan; // çünkü alan değişkenini değiştirir!
    }
};
int main() {
```

```

Sinif* nesne=new Sinif();
std::cout<<"nesne.alan:" << nesne->getAlan() << std::endl;
nesne->setAlan(12);
std::cout<<"nesne.alan:" << nesne->getAlan() << std::endl;
delete nesne;
const Sinif* constNesne=new Sinif();
//constNesne->setAlan(9); // Hata: const nesne (immutable object) değiştirilemez!
delete constNesne;
}

```

11.17 mutable Depolama Sınıfı

Bazen, `const` ile sabit olarak tanımlanmış nesne veya yapının (`struct`) bir veya daha fazla veri üyesini değiştirme gereksinimi doğabilir. İşte bu durumda bu üyeler `mutable` anahtar kelimesiyle tanımlanır.

```

#include <iostream>
class Kisi {
public:
    unsigned long tcno;
    mutable int okulno;
    Kisi(){
        tcno = 43233445505UL;
        okulno = -1;
    }
};
int main(){
    const Kisi ogrenci1; // ogrenci1 nesnesi const olarak tanımlandı
    ogrenci1.okulno = 2000; // sabit olan nesnenin okulno değiştiriliyor.
    std::cout << ogrenci1.okulno << std::endl;
    //ogrenci1.tcno = -1; //HATA: sabit nesnenin alanı değiştirilemez.
    /* assignment of member 'Kisi::tcno' in read-only object*/
}

```

11.18 Geçersiz Kılma

Geçersiz kılma (overriding), nesne yönelimli programlamada taban sınıfta önceden tanımlanmış bir yöntemi türemiş bir sınıfta yeniden tanımlamasına olanak tanıyan bir kavramdır. Yani taban sınıfın davranışının türemiş sınıfta değiştirilmesidir. C++ dili iki tür fonksiyon geçersiz kılmaı destekler; Birincisi derleme zamanı geçersiz kılma, diğeri ise çalışma zamanı geçersiz kılmaıdır.

Derleme zamanında geçersiz kılma aşağıdaki şekilde yapılır;

```

class Taban-Sınıf-Kimliği {
erişim-belirleyici:
    // geçersiz kılınacak yöntem:
    veri-tipi yöntem-kimliği() {
    }
};

class Türemiş-Sınıf-Kimliği: erişim-belirleyici Taban-Sınıf-Kimliği {
erişim-belirleyici:
    // geçersiz kılan yöntem:
    veri-tipi yöntem-kimliği() {
    }
};

```

Her iki sınıf ta da yöntemin kimliği aynıdır. Taban sınıftan imal edilen nesne taban sınıfın yöntemini, türemiş sınıftan imal edilen nesne ise türemiş sınıfın yöntemini icra eder.

```

#include <iostream>
class Baba {
public:
    void yap() {
        std::cout << "Baba şu şekilde yapar ..." << std::endl;
    }
};

```



```

class Cocuk : public Baba {
public:
    void yap() {
        std::cout << "Çocuk şu şekilde yapar..." << std::endl;
    }
};

int main() {
    Baba baba;
    baba.yap(); // Baba şu şekilde yapar ...
    Cocuk cocuk;
    cocuk.yap(); // Çocuk şu şekilde yapar...
}

```

Çalışma zamanında geçersiz kılma aşağıdaki şekilde yapılır;

```

class Taban-Sınıf-Kimliği {
    erişim-belirleyici:
    // geçersiz kılınacak yöntem:
    virtual veri-tipi yöntem-kimliği() {
    }
};

class Türemiş-Sınıf-Kimliği: erişim-belirleyici Taban-Sınıf-Kimliği {
    erişim-belirleyici:
    // geçersiz kılan yöntem:
    veri-tipi yöntem-kimliği() override {
    }
};

```

Her iki sınıf ta da yöntemin kimliği aynıdır. Geçersiz kılınacak yöntem, taban sınıfın yöntemini türemiş sınıfta değiştirilebileceğini belirtmek için `virtual` sıfatı () ile tanımlanır. Türemiş sınıfta ise geçersiz kılacağı yöntemi `override` belirleyicisi () ile yeniden tanımlar.

```

#include <iostream>
class Baba {
public:
    virtual void yap() {
        std::cout << "Baba şu şekilde yapar ..." << std::endl;
    }
};

class Cocuk : public Baba {
public:
    void yap() override {
        std::cout << "Çocuk şu şekilde yapar..." << std::endl;
    }
};

int main() {
    Cocuk cocuk;
    cocuk.yap(); // Çocuk şu şekilde yapar...
    Baba* babaPtr=&cocuk;
    babaPtr->yap(); // Çocuk şu şekilde yapar...
    /*
    Baba& babaPtr=cocuk;
    babaPtr.yap(); // Çocuk şu şekilde yapar...
    */
}

```

11.19 Arayüzler

Arayüzler (**interface**) nesnelerin sahip oldukları rollerdir (**role**). Roller bir grup davranışı (**behavior**) barındırır. C++ dilinde roller, sınıf olarak kodlanır. Ancak saf nesne yönelimli dillerde arayüzler, **interface** saklı kelimesiyle tanımlanırlar.

Örnek olarak bir öğretmen; Evde baba rolünde olup babanın sahip olduğu çocuklara bakma, yemek yapma gibi davranışları gösterir. İşte öğretmen rolündedir ve öğrenmenin sahip olduğu; öğretme, sınav

yapma, değerlendirme, karne hazırlama, ders notu hazırlama gibi davranışları vardır. Bu öğretmen müzisyen rolünde ise gitar çalma, akort yapma gibi davranışları da vardır. İşte bunların hepsi ayrı bir roldür ve her bir rolde o role ilişkin davranışları sergiler.

Arayüz Gerçekleştirme

Java, C# gibi saf nesne yönelimli dillerin aksine C++ Dilinde, **çoklu kalıtım (multiple inheritance)** desteklenir. Bir sınıfın arayüzleri miras almasına **arayüz gerçekleştirme (interface realization)** yada **arayüz uygulaması (interface implementation)** adı verilir.

Arayüzler, soyut (**abstract**) sınıflarda tanımlanır ve veri soyutlaması (**data abstraction**) yapılmaz. Bir sınıfın soyut olabilmesi için en az bir **saf sanal yöntem (pure virtual method)** sahip olmalıdır. Sanal sınıflardan nesne imal edilemez! Sanal sınıflardaki sanal yöntemler türeyen sınıfta yeniden tanımlanır ve mutlaka geçersiz kılınır (**override**).

```
#include <iostream>
#include <string> // Metinler başlığında detaylandırılacaktır.
class IBaba { //Baba Rolündeki davranışlar:
public:
    virtual void cocugaBak() = 0; // saf sanal yöntem-pure virtual method
    virtual void hanimaYardimEt() = 0; // saf sanal yöntem-pure virtual method
    /* Bu sınıfta en az bir saf sanal yöntem olduğundan bu sınıf soyut sınıftır olur. Bu nedenle bu
       ↳ sınıftan nesne imal edilemez! */
};

class IMuzisyen { //Müzisyen Rolündeki Davranışlar:
public:
    virtual void gitarCal() = 0; // saf sanal yöntem-pure virtual method
    virtual void piyanoCal() = 0; // saf sanal yöntem-pure virtual method
    /* Bu sınıfta en az bir saf sanal yöntem olduğundan bu sınıf soyut sınıftır olur. Bu nedenle bu
       ↳ sınıftan nesne imal edilemez! */
};

class IOgretmen { //Öğretmen Davranışları:
public:
    virtual void ogret() = 0; // saf sanal yöntem-pure virtual method
    virtual void dersNotuHazirla() = 0; // saf sanal yöntem-pure virtual method
    /* Bu sınıfta en az bir saf sanal yöntem olduğundan bu sınıf soyut sınıftır olur. Bu nedenle bu
       ↳ sınıftan nesne imal edilemez! */
};

class Kisi: public IBaba, public IMuzisyen, public IOgretmen {
    /* (interface realization/implementation): Gerçekleştirilen IBaba, IMuzisyen ve IOgretmen
       ↳ arayüzlerinde saf sanal yöntemler olduğundan bu Kişi sınıfında hepsi geçersiz
       ↳ kılınmalıdır (override). */
public:
    std::string adi;
    std::string soyadi;
    void adiniSoyle() {
        std::cout << adi << " " << soyadi << std::endl;
    }
    // IBaba aracılığıyla miras alınan davranışlar:
    void cocugaBak() override {
        std::cout << "Çocuğa Bakıyorum..." << std::endl;
    }
    void hanimaYardimEt() override {
        std::cout << "Hanıma Yardım Ediyorum..." << std::endl;
    }
    // IMuzisyen aracılığıyla miras alınan davranışlar:
    void gitarCal() override {
        std::cout << "Gitar Çalıyorum..." << std::endl;
    }
    void piyanoCal() override {
        std::cout << "Piyano Çalıyorum..." << std::endl;
    }
}
```

```

// IOgretmen aracılığıyla miras alınan davranışlar:
void ogret() override {
    std::cout << "Öğretiyorum..." << std::endl;
}
void dersNotuHazirla() override {
    std::cout << "Ders notu hazırlıyorum..." << std::endl;
}
};
int main() {
    Kisi ilhan;
    ilhan.adi = "İlhan";
    ilhan.soyadi = "ÖZKAN";

    //imal edilen nesne tüm arayüzdeki davranışları gösterebilir:
    ilhan.adiniSoyle(); // İlhan ÖZKAN
    ilhan.ogret(); // Öğretiyorum...
    ilhan.gitarCal(); // Gitar Çalıyorum...
    ilhan.hanimaYardimEt(); // Hanıma Yardım Ediyorum...

    //imal edilen nesne sadece bir roldeki davranışları da gösterebilir:
    IBaba& baba=ilhan;
    baba.cocugaBak(); // Çocuğa Bakıyorum...
    baba.hanimaYardimEt(); // Hanıma Yardım Ediyorum...

    IMuzisyen& muzisyen = ilhan;
    muzisyen.gitarCal(); // Gitar Çalıyorum...
    muzisyen.piyanoCal(); // Piyano Çalıyorum...

    IOgretmen& ogretmen = ilhan;
    ogretmen.dersNotuHazirla(); // Ders notu hazırlıyorum...
    ogretmen.ogret(); // Öğretiyorum...
}

```

11.20 Çok Biçimlilik

Çok biçimlilik (polymorphism), birden çok nesnenin olduğu bir ortamda, **verilen aynı iletiye (message) karşı, nesnelerin farklı davranış (behavior) göstermeleridir (same message-different behavior)**.

Çok Biçimliliğin Uygulaması

Çok biçimliliğin kodlamada uygulanabilmesi (**implementation**) için üç şey gereklidir;

1. **Kalıtım (inheritance)**: Ortak davranışın tanımlandığı taban sınıf ve bu davranışın değiştiği türeyen sınıflar.
2. **Sanal Yöntem (virtual method)**: Nesnelere aynı iletiyi verebilmek için ortak davranış taban sınıfta tanımlayan sanal yönetime sahip olmalıdır.
3. **Geçersiz Kılma (override)**: Türemiş sınıfta devralınan davranış kendine uyarlayan ve taban sınıftaki sanal yöntemi geçersiz kılan yeni bir yöntem.

```

#include <iostream>
#include <string>
class SoyutKisi { //Soyut Kisi Sınıfı
public:
    /* Sanal Yıkıcı (Virtual Destructor): Eğer temel sınıfın yıkıcısı sanal olmazsa, delete
    → kisi[i] dendiğinde türemiş sınıfların (Ogrenci, Mudur vb.) özel alanları
    → temizlenmeyebilir.*/
    virtual ~SoyutKisi() = default;
    virtual void raporYaz() = 0;
    /* saf sanal yöntem: Bu yöntem, Kişi sınıfını SOYUT yapar. Ayrıca; raporYaz(): Türeyen
    → sınıflardan imal edilecek nesnelere aynı mesajı vermek için tanımlanan ortak davranıştır.
    → */
    std::string getAdi() {
        return adi;
    }
}

```

```

    }
    void setAdi(std::string pAdi) {
        if (pAdi!="") this->adi=pAdi;
    }
    SoyutKisi(std::string pAdi): adi(pAdi) {
    }
    private:
    std::string adi;
};

class Ogrenci: public SoyutKisi { //Ortak davranış SoyutKisi sınıfından devralınıyor.
public:
    void raporYaz() override { //devralınan yöntemler geçersiz kılınıyor (override)
        std::cout << getAdi() <<"Öğrenci Ödevlerine İlişkin Rapor Yazıyor..." << std::endl;
    }
    Ogrenci(std::string pOgrenciAdi):SoyutKisi(pOgrenciAdi){
    }
};

class Ogretmen: public SoyutKisi { //Ortak davranış SoyutKisi sınıfından devralınıyor.
public:
    void raporYaz() override { //devralınan yöntemler geçersiz kılınıyor (override)
        std::cout << getAdi() <<"Ogretmen Verdiği Derslere İlişkin Rapor Yazıyor..."
        << std::endl;
    }
    Ogretmen(std::string pOgretmenAdi):SoyutKisi(pOgretmenAdi){
    }
};

class Mudur: public SoyutKisi { //Ortak davranış SoyutKisi sınıfından devralınıyor.
public:
    void raporYaz() override { //devralınan yöntemler geçersiz kılınıyor (override)
        std::cout << getAdi() <<"Müdür Akademik Takvime İlişkin Rapor Yazıyor..."
        << std::endl;
    }
    Mudur(std::string pMudurAdi):SoyutKisi(pMudurAdi){
    }
};

int main() {
    /* Modern C++'ta çıplak diziler yerine std::vector ve smart pointer kullanılır.
    → std::unique_ptr sayesinde 'delete' yazmanıza gerek kalmaz, bellek sızıntısı önlenir.
    → İleride anlatılacaktır. */
    SoyutKisi* kisi[3];
    kisi[0]=new Ogrenci("İlhan ÖZKAN");
    kisi[1]=new Ogretmen("Hasan YILMAZ");
    kisi[2]=new Mudur("Recep AY");
    for (int i=0; i<3; i++)
        kisi[i]->raporYaz();
    /*
    Her kişiye aynı "raporYaz()" mesajı veriliyor->
    Karşılığında farklı davranışlar sergileniyor.
    */
    delete kisi[0];
    delete kisi[1];
    delete kisi[2];
}

/* Program Çıktısı:
İlhan ÖZKAN:Öğrenci Ödevlerine İlişkin Rapor Yazıyor...
Hasan YILMAZ:Ogretmen Verdiği Derslere İlişkin Rapor Yazıyor...
Recep AY:Müdür Akademik Takvime İlişkin Rapor Yazıyor...
*/

```

Çok Biçimlilikte Hangi Yöntem Ne Zaman Çalıştırılır?

Her sanal (**virtual**) yönteme sahip nesnenin gizli bir **vptr** göstericisi taşır. Derleyici bu göstericilerden bir **vTable** tablosu oluşturur. Çalışma zamanında bu **sanal yöntemler tablosu** (**virtual method table**) üzerinden doğru fonksiyon adresine dallanılır.

Çok biçimlilik bize çalışma anında (**run-time**) üstünlük sağlar. Bu nedenle nesneler de çalışma anında imal edilmiştir. Program çalıştırıldığında her **kisi** nesnesine aynı ileti verilmiş ama bu nesneler farklı davranış göstermiştir. Yazılımcı, nesnenin tipine bakmaksızın, nesnelerden aynı davranışı göstermesini ister. Burada gösterilecek davranışın, taban sınıfın davranışı mı? olduğu veya türeyen sınıfın davranışı mı? olduğuna çalışma anında (**run-time**) karar verilir. İşte bu karar verme işlemine **geç bağlama** (**late binding**) veya **dinamik bağlama** (**dynamic binding**) adı verilir. Esnekliğin istenmediği bazı durumlarda ise karar verme işlemi derleme anında (**compile-time**) yapılır. Buna ise **statik bağlama** (**static binding**) adı verilir. İkisi arasındaki seçim programcı tarafından yapılır.

Yazılım yapılacak sistemin başlangıç koşullarına bağlı tasarım kararlarının tümü bilinemez. Ayrıca **sis-temin genişlemesi için gerekli tüm kodlamanın bitirilmesi beklenemez**. İşte bunu gerçekleştiren **dinamik bağlama**, yazılım mimarisinin daha esnek (mevcut bileşenin sistemde yeniden yapılandırmasının kolayca yapılması) ve genişleyebilir (yeni bileşenlerin kolayca sisteme eklenmesi) olmasını sağlar.

11.21 Sınıf Çeşitleri

Soyut Sınıf (**abstract class**): Bu sınıfların bir örneği (**instance**) olamaz. Yani bu sınıftan nesne imal edilemez. Bu sınıfın en az bir soyut yöntemi olur. Soyut yöntemler, **saf sanal yöntemlerdir** (**pure virtual method**). Soyut sınıflar hangi davranışın gösterileceğini belirler ama bu davranışların nasıl gösterileceğini anlatmaz. Bu nedenle soyut yöntem için gövde tanımlanmaz.

```
#include <iostream>
#include <string>
class SoyutKisi { //Soyut Kisi Sınıfı
public:
    virtual void raporYaz() = 0; // Bu saf sanal yöntem sınıfı soyut yapar.
    std::string getAdi() {
        return adi;
    }
    void setAdi(std::string pAdi) {
        if (pAdi!="") this->adi=pAdi;
    }
    SoyutKisi(std::string pAdi): adi(pAdi) {}
private:
    std::string adi;
};
int main() {
    // SoyutKisi soyutkisi; //HATA: Soyut sınıftan nene imal edilemez!
}
```

Somut sınıf (**concrete class**): Bu sınıflardan nesne imal edilebilir yani örneği (**instance**) olabilir. Bu sınıflarda davranış ve durumlar eksiksiz tanımlıdır. Bir sınıf tanımlarken soyut bir sınıfa genel davranış ve durumları toplamak ve ondan miras alarak somut sınıfları tanımlamak her zaman iyi bir seçimdir.

```
#include <iostream>
#include <string>
class SoyutKisi { //Soyut Kisi Sınıfı
public:
    virtual void raporYaz() = 0; // Bu saf sanal yöntem sınıfı soyut yapar.
    std::string getAdi() {
        return adi;
    }
    void setAdi(std::string pAdi) {
        if (pAdi!="") this->adi=pAdi;
    }
    SoyutKisi(std::string pAdi): adi(pAdi) {}
private:
    std::string adi;
};
```

```

};
class SomutKisi: public SoyutKisi { //Ortak davranışlar SoyutKisi sınıfından devralınıyor.
public:
    void raporYaz() override {
        std::cout << getAdi() <<" : Rapor Yazıyor..." << std::endl;
    }
    // Bu sınıfta sanal yöntem olmadığından nesne imal edilebilir.
    SomutKisi (std::string pAdi):SoyutKisi(pAdi){}
};
int main() {
    SomutKisi ilhan("Ilhan");
    ilhan.raporYaz(); // "Ilhan: Rapor Yazıyor..."
}

```

Taban sınıf (base class): Bu sınıf, miras alınan sınıf veya genel özellik ve davranışların toplandığı yani genelleştirmenin yapıldığı **genel sınıf (general class)** veya **ebeveyn sınıftır (parent class)**.

Türemiş sınıf (derived class): Bu sınıf, miras alan sınıf veya davranış ve durumlar hakkında uzmanlaşmış yani **uzman sınıf (special class)** veya **çocuk sınıftır (child class)**. Türemiş ya da miras almış sınıf.

Kök sınıf (root class): Bu sınıf, babası olmayan ya da yetim sınıf olup aynı zamanda baba sınıftır.

Yaprak sınıf (leaf class): Bu sınıf, kendisinden henüz miras alan bir sınıf olmayan bir sınıftır. Kısaca türememiş sınıftır.

Tekil sınıf (singleton class): Bu sınıftan yalnızca bir nesne imal edilebilir. Yani yalnızca bir örneği vardır. Bunun olabilmesi için mahrem (**private**) yapıcısı olması gerekir. İleride incelenecektir.

Yardımcı sınıf (utility class): Bu sınıf, üyeleri statik olan sınıflardır. Ayrıca evrensel yapılandırma ayarlarını (**configuration settings**) veya değişmezleri (**literal**) depolamak için kullanılabilir. Bir kaynak havuzunu (önbellek, veri tabanı bağlantı havuzu, vb.) yönetmek ve örnekler arasında paylaşılan bir günlük sistemi uygulamak için yararlıdır. Bunların dışında, izleme yöntemi (**trace method**) çağrılarını için de kullanılabilir.

Kısır sınıf (sealed class): Bu sınıf, kendisinden türeyen bir sınıf tanımlayamayan bir sınıftır. Kısaca türeyemeyen bir sınıftır. C++ diline 2011 yılında bunu yapabilmek için **final** anahtar kelimesi eklenmiştir.

```

#include <iostream>
class Dikdortgen final { //Somut Dikdörtgen Sınıfı
public:
    virtual float alanHesapla() {
        return en*boy;
    };
    Dikdortgen(float pEn, float pBoy): en(pEn), boy(pBoy) {}
private:
    float en, boy;
};
class Kare: public Dikdortgen { //HATA: Dikdörtgen sınıfı final ile tanımlanmış
public:
    Kare(float pEn) : Dikdortgen(pEn, pEn) {}
};
int main() {
    Dikdortgen dikdortgen(4.0,5.0);
    std::cout << "Dikdörtgenin alanı:" << dikdortgen.alanHesapla() << std::endl;
}

```

final anahtar kelimesi bir sınıfta sadece yöntemler için de kullanılabilir. Bu durumda türeyen sınıfta bu yöntem geçersiz kılınamaz (override);

```

#include <iostream>
class Dikdortgen { //Somut Dikdörtgen Sınıfı
public:
    virtual float alanHesapla() {
        return en*boy;
    };
    virtual float kisaKenar() final {
        return (en>boy)?boy:en;
    }
    Dikdortgen(float pEn, float pBoy): en(pEn), boy(pBoy) {}
}

```

```

private:
    float en, boy;
};
class Kare: public Dikdortgen {
public:
    /*
    float kısaKenar() override { //HATA: Dikdörtgen sınıfında kısaKenar yöntemi final ile
        ↳ tanımlanmış
        //...
        return 0.0f;
    }
    */
    Kare(float pEn) : Dikdortgen(pEn, pEn) {
    }
};
int main() {
    Dikdortgen dikdortgen(4.0,5.0);
    std::cout << "Dikdörtgenin kısa kenarı:" << dikdortgen.kisaKenar() << std::endl;
}

```

11.22 Dinamik Bellek Yönetiminde Kopyalama Tehlikeleri

Sınıf (**class**) yapılarının içerisinde **new** anahtar kelimesi kullanılarak dinamik olarak tahsis edilen (Heap bellek bölgesinde tutulan) üye değişkenler bulunduğunda, C++'ın varsayılan davranışları büyük bir güvenlik tehdidine dönüşür. Derleyicinin bizim yerimize otomatik olarak ürettiği **varsayılan kopyalama yapıcısı** (**default copy constructor**) ve **varsayılan atama işleci** (**default assignment operator**), dinamik bellek barındıran sınıflarda programın beklenmedik şekilde davranmasına, **bellek sızıntılarına** (**memory leak**) ve en nihayetinde aynı belleğin **iki kez serbest bırakılması** (**double free**) hatasıyla programın çökmesine (**crash**) neden olur.

C++ derleyicisi, biz aksini belirtmediğimiz sürece bir nesne bir başka nesneye kopyalanırken (veya fonksiyona değer olarak gönderilirken) **sığ kopyalama** (**shallow copy**) yapar. Sığ kopyalamada, nesnenin tüm alan değişkenleri bit bit (birebir) kopyalanır.

Eğer alan değişkeni bir gösterici (**pointer**) ise, işaretçinin işaret ettiği adres kopyalanır; adresteki verinin kendisi kopyalanmaz. Aşağıdaki **DinamikDizi** sınıfı, içinde dinamik bir tam sayı dizisi barındırmaktadır ancak kopya yapıcısı ve atama işleci yazılmamıştır.

```

#include <iostream>

class DinamikDizi {
public:
    int* veri;
    int boyut;

    // Yapıcı (Constructor)
    DinamikDizi(int b) {
        boyut = b;
        veri = new int[boyut]; // Heap bölgesinde bellek tahsisi
        for(int i = 0; i < boyut; i++) veri[i] = i * 10;
        std::cout << "Kurucu: " << boyut << " elemanlık yer ayrıldı.\n";
    }

    // Yıkıcı (Destructor)
    ~DinamikDizi() {
        delete[] veri; // Tahsis edilen alanı temizleme
        std::cout << "Yıkıcı: Bellek serbest bırakıldı.\n";
    }
};

```

Yukarıdaki sınıfı kullanan bir main fonksiyonu yazalım;

```

int main() {
    DinamikDizi nesne1(3); // 1. Adım
    DinamikDizi nesne2 = nesne1; // 2. Adım (Varsayılan Kopyalama Yapıcısı tetiklenir)
    return 0; // 3. Adım (main biterken yıkıcı fonksiyonlar çağrılır)
}

```


}
Program çalıştığında arka planda yığın (**stack**) ve öbek (**heap**) bellek bölgelerinde ne olduğunu inceleyelim;

- ◊ Birinci adımda `nesne1` için yığın bölgesinde yer açılır. `veri` isimli gösterici, öbek bölgesinde `new` ile tahsis edilen `0x00A1` adresini göstermektedir.
- ◊ İkinci adımda; derleyici, el ile yazılmış bir kopyalama yapıcısı bulamadığı için varsayılan sıg kopyalamayı çalıştırır. `nesne1`'in içindeki değerleri doğrudan `nesne2`'ye kopyalar. `nesne2.boyut = nesne1.boyut`; yani 3 olur. `nesne2.veri = nesne1.veri`; olacağından `nesne2.veri` değeri `0x00A1` olacaktır yani adresi doğrudan kopyalanır!
- ◊ Artık öbek bölgesindeki aynı tek veriyi (`0x00A1`) gösteren iki farklı nesnemiz var. Nesnelerden biri bu veriyi değiştirirse diğeri de farkında olmadan değişmiş veriyi okur. Bu durumda **kritik tehlike başlamıştır** ve **veri tutarsızlığı** yaşanabilir.
- ◊ Üçüncü adımda; yani `main` fonksiyonu sonlandığında, yığındaki yerel nesneler oluşturulma sırasının tersine göre yok edilir. Yani önce `nesne2`'nin, ardından `nesne1`'in yıkıcısı (`DinamikDizi()`) çağrılır. İlk önce `nesne2`'nin Yıkıcısı Çalışır: `delete[] veri`; talimatı verilir. `nesne2`'nin verisi `0x00A1` adresini gösterdiği için, öbek üzerindeki `0x00A1` adresi işletim sistemine iade edilir. Bu alan artık boşadır. Daha sonra `nesne1`'nin Yıkıcısı Çalışır: `delete[] veri`; talimatı verilir. `nesne1`'in verisi de hala `0x00A1` adresini göstermektedir. Ancak bu adres az önce `nesne2` tarafından zaten serbest bırakılmıştır! İşletim sistemi, zaten serbest bırakılmış (artık kendisine ait olmayan veya başka bir işleme tahsis edilmiş olabilecek) bir bellek adresini ikinci kez silmeye çalıştığımızı fark eder. Program anında çalışmayı durdurur ve konsola şu ölümcül hatayı basar:
`free(): double free detected in tcache 2`
`Aborted (core dumped)`

Eğer nesneler ilk oluşturulma anında kopyalanmıyor da, daha sonra birbirine atanıyorsa (`nesne2 = nesne1`);, bu sefer hem çifte serbest bırakma (**double free**) hatası hem de bellek sızıntısı (**memory leak**) oluşur. Bu durumu gözlemek için ana fonksiyonumuzu aşağıdaki gibi değiştirelim ve çalıştırınca neler olduğuna bakalım;

```
int main() {
    DinamikDizi nesne1(3); // Heap'te 0x00A1 adresinde yer açıldı.
    DinamikDizi nesne2(5); // Heap'te 0x00B2 adresinde yer açıldı.

    nesne2 = nesne1; // Varsayılan Atama Operatörü çalışır (Sığ kopyalama)
    return 0;
}
```

Burada `nesne2 = nesne1`; satırı çalıştığında, `nesne2.veri` göstericisinin değeri de `0x00A1` olur. `nesne2`'nin daha önce tuttuğu `0x00B2` adresindeki 5 elemanlık eski bellek alanına giden köprü kopmuştur. O belleği silecek hiçbir gösterici kalmamıştır. Program çalıştığı sürece o bellek RAM'de fuzuli yer kaplayarak bellek sızıntısına yol açar. `main` fonksiyonu biterken yine hem `nesne1` hem `nesne2` aynı `0x00A1` adresini `delete` etmeye çalışacak ve program yine çökecektir.

Üçler Kuralı

Bu felaketi önlemenin tek yolu derin kopyalama (**deep copy**) ve üçler kuralını (**rule of three**) uygulamaktır. Sınıfın içinde derin kopyalama mekanizmasını el ile kurulur. C++ dünyasındaki klasik üçler kuralı der ki: **Eğer bir sınıfın yıkıcısına (destructor) ihtiyacı varsa, büyük ihtimalle kendi kopyalama yapıcısına ve atama operatörüne de ihtiyacı vardır.**

Bu durumda doğru sınıf tasarımı aşağıda verilmiştir;

```
class DinamikDizi {
public:
    int* veri;
    int boyut;

    DinamikDizi(int b) {
        boyut = b;
        veri = new int[boyut];
    }

    ~DinamikDizi() {
```



```

        delete[] veri;
    }

    // 1. KOPYALAMA YAPICISI (Copy Constructor) - DERİN KOPYALAMA
    DinamikDizi(const DinamikDizi& kaynak) {
        boyut = kaynak.boyut;
        // Sadece adresi kopyalamıyoruz, Heap'te yeni bir alan açıyoruz!
        veri = new int[boyut];
        // Kaynağın içindeki verileri tek tek yeni açtığımız alana kopyalıyoruz
        for (int i = 0; i < boyut; i++) {
            veri[i] = kaynak.veri[i];
        }
        std::cout << "Kopyalama Yapicisi: Derin kopyalama yapildi.\n";
    }

    // 2. ATAMA İŞLECI (Assignment Operator) - DERİN KOPYALAMA
    DinamikDizi& operator=(const DinamikDizi& kaynak) {
        // Kendine atama işleci (Örn: n1 = n1; durumu)
        if (this == &kaynak) return *this;

        // Önce kendi üzerimizdeki eski belleği temizlemeliyiz (Sızıntıyı önler)
        delete[] veri;

        // Şimdi derin kopyalamayı gerçekleştiriyoruz
        boyut = kaynak.boyut;
        veri = new int[boyut];
        for (int i = 0; i < boyut; i++) {
            veri[i] = kaynak.veri[i];
        }
        std::cout << "Atama Operatoru: Eski bellek silindi, yeni derin kopyalama yapildi.\n";
        return *this;
    }
};

```

Sınıfların Alan Değişkenlerinde Gösterici Kullanımı

Bir C++ mühendisi olarak, sınıf tasarlarken üye değişkenler arasında ham gösterici (**raw pointer**) ve **new/delete** kullanımı mevcutsa, derleyicinin arka planda yapacağı sığ kopyalama işlemlerinin programı çökerteceğini asla unutmamalısınız. Modern C++ mimarisinde bu riskleri sıfıra indirmek için ham göstericiler yerine sonraki bölümlerde göreceğimiz Akıllı göstericiler (**smart pointer**) olan **std::unique_ptr**, **std::shared_ptr** veya **std::vector** gibi hazır STL konteynerleri tercih edilmelidir.

11.23 Numaralandırma Sınıfı

Özel bir sınıf olan **numaralandırma Sınıfı** (**enumeration class**), C++ dilinde **kapsamlı numaralandırma** (**scoped enumeration**) olarak da adlandırılır. Bir grup tam sayıdan oluşan değişmezden (**integral literal**) oluşan numaralandırılmış kullanıcı tanımlı bir veri tipidir. Bir bütünün parçalarını tam sayılar olarak ifade eden değişmezlere kullanıcı tanımlı adlar atamak istediğinizde kullanışlıdır.

```

enum class numaralandirmakimligi {
    adlandırılmış-tam-sayı1 [=DEĞER1],
    adlandırılmış-tam-sayı2 [=DEĞER2],
    ...
};

```

Sözde kodda DEĞER1, DEĞER2 olarak belirtilenler aslında tam sayı değişmezleridir. Yani tam sayı değişmezlerdir (**integer literal**). Bu değişmezler büyük harfle yazılırlar ve belirtilmedikçe ilkinin değeri 0, ikincisinin değeri 1, ... şeklinde ilerler.

```

enum class Durum {
    OK = 0,        // 0
    HATA = 1,      // 1
    UYARI = 2      // 2
};

```

```
};

enum class Cinsiyet {
    BELIRTILMEMIS, // 0
    ERKEK,         // 1
    KADIN          // 2
};
```

Bu numaralandırma sınıfı öğelerine bir kod içerisinde yine **kapsam çözümleme işleci** (**scope resolution operator**) (`::`) ile erişilir. İstenirse tam sayılardan oluşan numaralandırma `int` yerine başka bir tam sayı veri tipinde (`char`, `unsigned`, `short`, ...) de oluşturulabilir. Bu durumda numaralandırma sınıfı aşağıdaki şekilde tanımlanır.

```
enum class numaralandirmakimligi: ferkli-bir-tamsayı-veritipi {
    adlandırılmış-tam-sayı1 [=DEĞER1],
    adlandırılmış-tam-sayı2 [=DEĞER2],
    ...
};
```

Cinsiyet örneğimizi aşağıdaki şekilde de yazabiliriz;

```
enum class cinsiyetKodu:char {
    BELIRSIZ = 'B',
    ERKEK = 'E',
    KADIN = 'K'
};
```

Farklı bir tiple numaralandırma sınıfı oluşturulmuş ise kullanılırken `static_cast` işleci ile tip dönüşümü yapılmalıdır. Tip dönüşümleri ileride anlatılacaktır. Aşağıda buna ilişkin bir örnek verilmiştir.

```
#include <iostream>
#include <string>
enum class Cinsiyet : char {
    BELIRSIZ = 'B',
    ERKEK = 'E',
    KADIN = 'K'
};

struct Ogrenci { // C++ dilinde struct ile class arasında çok bir fark bulunmaz!
    unsigned int yas;
    Cinsiyet cinsiyet;
    float kilo;
    unsigned int boy;

    void bilgileriYazdir(const std::string& etiket) const {
        std::cout << etiket << ":" << std::endl
            << "Yaşı: " << yas
            << ", Cinsiyeti: " << static_cast<char>(cinsiyet) // Enum değerini char olarak
            << " yazdırır"
            << ", Kilosu: " << kilo
            << ", Boyu: " << boy << std::endl;
    }
};

int main() {
    // Nesne oluşturma ve atama
    Ogrenci ogrenci1;
    ogrenci1.yas = 19;
    ogrenci1.cinsiyet = Cinsiyet::ERKEK;
    ogrenci1.kilo = 75.5f;
    ogrenci1.boy = 180u;
    ogrenci1.bilgileriYazdir("Öğrenci 1");

    Ogrenci ogrenci2 = {25, Cinsiyet::KADIN, 55.0f, 165u};
    ogrenci2.bilgileriYazdir("Öğrenci 2");
}

/*Program Çıktısı:
Öğrenci 1:
```

```
Yaşı: 19, Cinsiyeti: E, Kilosu: 75.5, Boyu: 180
Öğrenci 2:
Yaşı: 25, Cinsiyeti: K, Kilosu: 55, Boyu: 165
*/
```

11.24 Düşün ve Çöz

- ◇ **Düşün:** Aşırı yükleme (**overloading**) ve varsayılan argümanlar (**default arguments**) ne zaman kullanılmalıdır? İkisini aynı anda kullanmak belirsizliğe (**ambiguity**) neden olabilir mi? Örnek verin.
- ◇ **Düşün:** Kapsülleme (**encapsulation**), kalıtım (**inheritance**) ve çok biçimlilik (**polymorphism**) ilkelerini gerçek dünyadan birer örnekle açıklayın. Bu üç ilke birbiriyle nasıl ilişkilidir?
- ◇ **Çöz:** Yapıcı (**constructor**) aşırı yüklemesinin kullanıldığı bir 'Dikdörtgen' sınıfı tasarlayın. Parametresiz yapıcı, kenar uzunlukları alan yapıcı ve kopya yapıcı tanımlayın. Nesne yıkıcıyı (**destructor**) ne zaman yazmak zorunlu hale gelir?
- ◇ **Düşün:** Sanal yöntem (**virtual method**) ile sanal olmayan yöntem arasındaki farkı veri tablosu (**vtable**) mekanizması üzerinden açıklayın. Çalışma zamanı çok biçimliliği (**runtime polymorphism**) bu mekanizma olmadan mümkün olur muydu?
- ◇ **Düşün:** Soyut sınıf (**abstract class**) ile arayüz (**interface**) kavramını karşılaştırın. C++'da arayüz nasıl simüle edilir? Saf sanal fonksiyon (= 0) bildiriminin pratik etkisi nedir?
- ◇ **Çöz:** **this** göstericisinin amacını açıklayın. Bir yöntem içinde **return *this;** talimatını döndürmenin ne işe yaradığını, zincirleme yöntem çağrısı (**method chaining**) örneğiyle gösterin.
- ◇ **Düşün:** Statik sınıf üyeleri **static** neden "sınıfa ait" olarak tanımlanır?